

Chapter

08

지금까지 우리가 만든 프로그램들은 대부분 클로드 코드가 제공하는 기본 기능 안에서 작동했습니다. 하지만 실제 개발 프로젝트에서는 기본 기능만으로는 부족할 때가 있습니다. MCP는 이런 한계를 넘어, 클로드 코드의 능력을 무한히 확장해 주는 강력한 도구입니다. 마치 개발 환경에 플러그인을 추가하듯 MCP를 연결하면 클로드 코드가 외부 시스템과 자유롭게 협력할 수 있습니다.

이번 장에서는 노션과 연동해 문서를 자동 작성하고, 깃허브에 프로젝트를 업로드하며, 데이터베이스를 연결해 실제로 동작하는 서비스를 만들어 봅니다. 로컬 환경에 머물던 애플리케이션이 이제 전 세계 누구나 접근할 수 있는 진짜 서비스로 성장하게 될 것입니다.

MCP로 클로드 코드의 한계 넘어서기

학습목표

- MCP의 개념을 이해하고 클로드 코드와 연결할 수 있습니다.
- 원격 MCP와 로컬 MCP 서버를 구분하고 필요에 맞게 설치할 수 있습니다.
- Playwright MCP로 웹 애플리케이션을 자동으로 테스트할 수 있습니다.
- 깃허브 MCP를 통해 프로젝트를 버전 관리하고 업로드할 수 있습니다.
- Vercel과 Supabase를 활용해 실제 서비스를 배포하고 운영할 수 있습니다.

08-1

MCP 이해하고 클로드 코드와 연결하기

핵심 키워드

MCP

클라이언트 서버 모델

로컬 MCP

원격 MCP

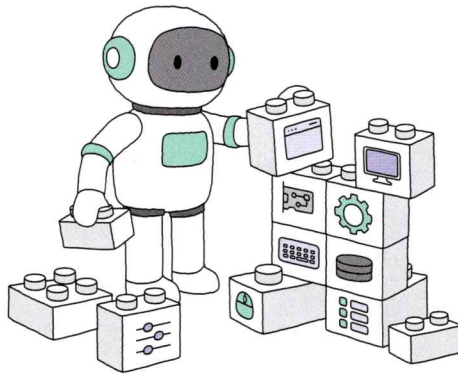
인증

--

이번 절에서는 MCP가 AI의 가능성을 어떻게 확장시키는지 살펴봅니다. 클로드 코드가 MCP를 통해 외부 서비스에 문서를 생성하거나, 로컬 MCP를 이용해 복잡한 문제를 단계적으로 추론하는 과정을 따라가며, AI가 단순한 응답을 넘어 실제 업무 도구와 연결되어 일하는 방식을 익혀 보겠습니다.

시작하기 전에

레고 블록을 조립할 때 기본 세트만으로는 한계가 있지만, 특별한 부품들을 추가하면 상상했던 모든 것을 만들 수 있게 되죠. MCP는 클로드 코드에 이런 특별한 부품들을 연결해 주는 연결 장치입니다. MCP는 클로드를 만든 앤트로픽 Anthropic 사가 2024년 11월에 제안한 이후, 현재는 AI를 개발하는 전 세계 개발자들의 표준이 되었습니다. 지금부터 이 강력한 도구들을 하나씩 연결해 보며, 클로드 코드의 진가를 직접 경험해 봅시다.

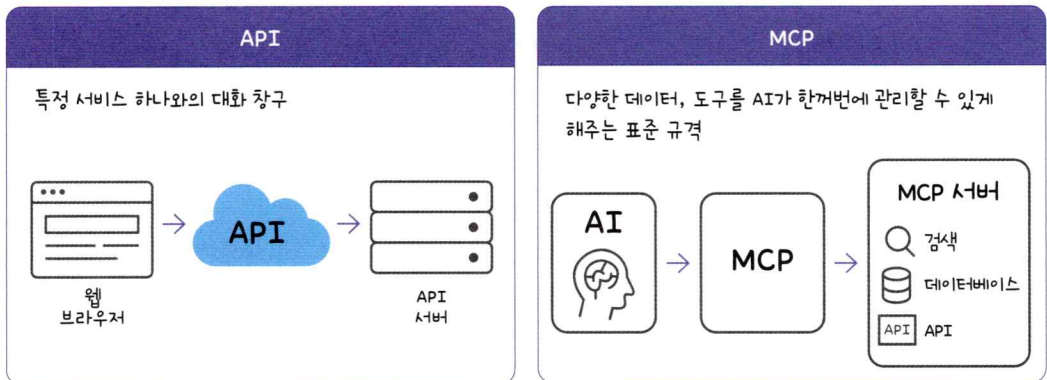


MCP가 열어준 AI의 무한 확장 가능성

MCP Model Context Protocol 는 AI 모델이 외부 데이터, 도구, 서비스와 연결될 때 사용하는 표준화된 통신 규약입니다. 즉 AI가 다양한 시스템과 일관된 방식으로 대화하고 협업할 수 있게 해주는 다리 역할 이죠.

API는 서로 다른 프로그램이 데이터를 주고받을 수 있게 하는 인터페이스로, 다른 서비스의 기능을 잠시 빌려오는 방식입니다. 실제로 ‘냉장고를 부탁해’ 애플리케이션을 제작하면서 오픈라우터의 API 키를 사용해 AI 모델을 가져와 이미지 인식과 텍스트 생성에 사용했죠. 반면 MCP는 여러 API와 파일, 데이터베이스를 한꺼번에 통합해 연결하는 프로토콜^{protocol}입니다. MCP를 통해 AI 모델은 여러 외부 서비스와 데이터 소스에 직접 접근하고 이를 활용할 수 있게 됩니다.

예를 들어 API를 활용해 AI 모델에 텍스트 생성을 요청할 수는 있지만, AI 모델이 파일 시스템을 직접 조작하거나 데이터베이스를 수정하게 할 수는 없습니다. 하지만 MCP를 사용하면 AI가 직접 파일 시스템에 접근하고, 데이터베이스를 조회하거나 수정할 수 있습니다.

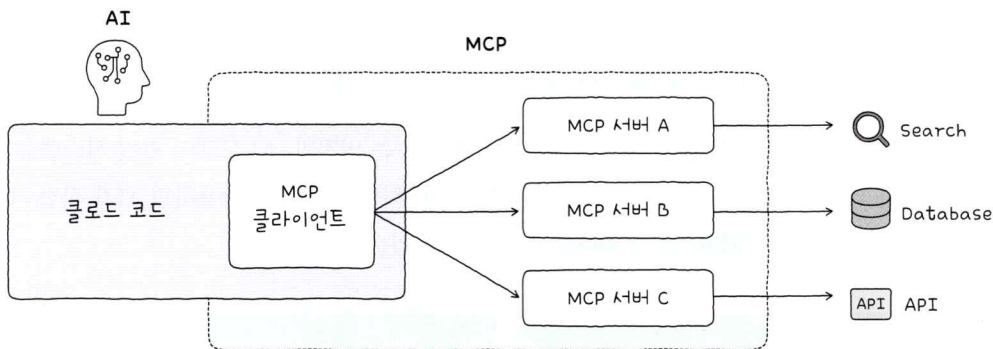


클로드 코드는 이미 MCP 클라이언트의 기능을 내장하고 있습니다. 사용자가 어떤 MCP를 사용할지 설정하면, 클로드 코드는 설정에 따라 MCP 서버에 작업을 요청합니다. 그러면 MCP 서버는 자신이 가진 기능을 이용해 실제 작업을 수행하죠.

예를 들어, ‘파일 탐색과 코드 분석’을 담당하는 MCP 서버를 연결하면 클로드 코드는 서버를 통해 시스템에 접근하여 폴더 구조를 살피고, 관련 코드를 분석하여 자동으로 보고서를 작성할 수 있습니다. 이처럼 MCP를 이용하면 클로드 코드가 다룰 수 있는 작업의 범위가 넓어집니다.

클라이언트-서버 모델 Client-Server Model, CS model은 네트워크에서 정보를 요청하고 응답하는 구조를 말합니다. 클라이언트는 서버를 요청하는 쪽이고, 서버는 서비스를 제공하는 쪽입니다. 즉, 클라이언트가 요청을 보내고 서버가 그 요청에 맞는 응답을 돌려주는 구조를 말합니다.

클로드의 MCP도 클라이언트-서버 모델을 따릅니다. 클로드가 클라이언트로서 요청을 보내고, MCP 서버가 실제 작업을 처리해 결과를 돌려주는 구조입니다. 이 모델의 장점은 두 가지입니다. 첫째, 새로운 기능을 추가할 때 MCP 서버만 연결하면 되므로 확장이 쉽습니다. 둘째, 클로드는 작업 지시만 하고 실제 실행은 각 MCP 서버가 담당하므로 안정성이 높습니다. 단, MCP 서버가 실제로 외부 파일이나 API에 접근하므로 보안 인증과 접근 권한 설정을 꼼꼼히 해야 합니다.



클로드 코드와 노션 연결하기

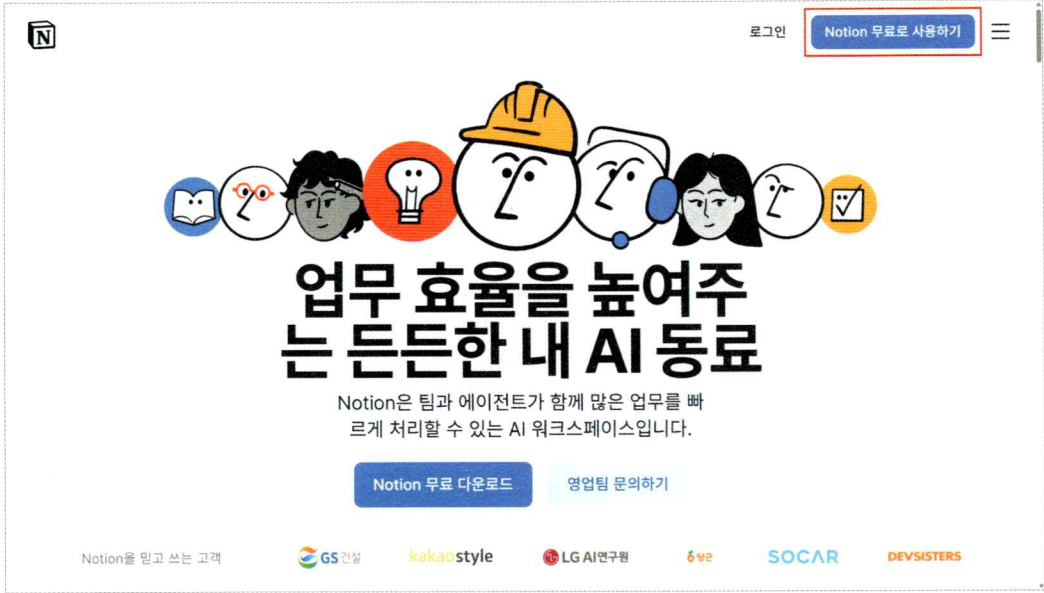
지금부터 클로드 코드에 실제로 MCP를 연결해서 실행해 보겠습니다. 이 책에서는 MCP를 사용해 문서를 작성하고 관리하는 협업 도구인 **노션** Notion에 클로드 코드를 연동해서 실제로 페이지를 생성하고 문서를 작성해 보겠습니다.

노션 가입하기

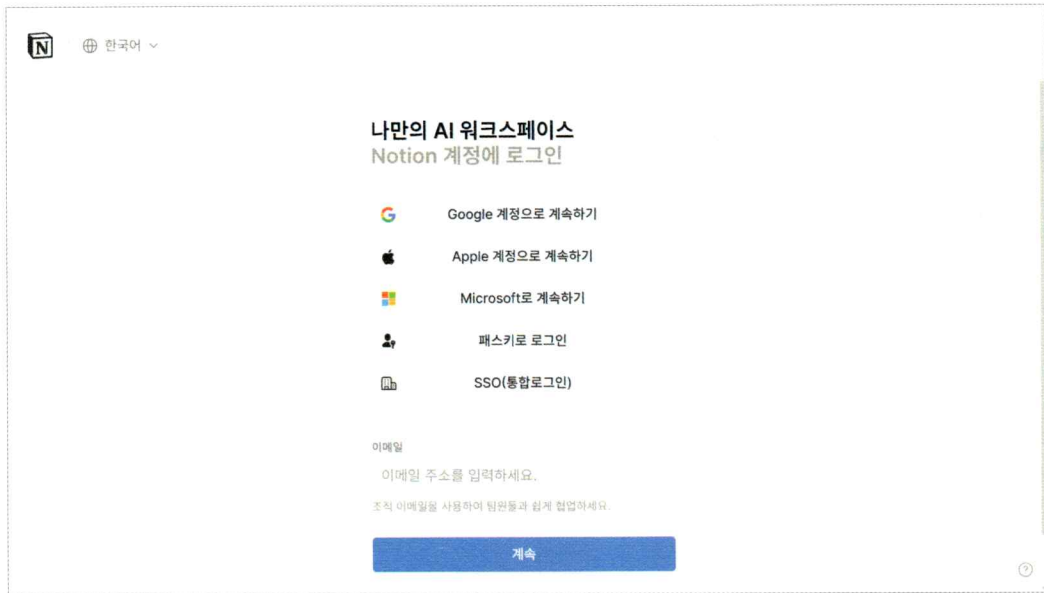
노션은 문서와 프로젝트를 생성 및 관리하거나 데이터베이스 자체로 사용되는 등 용도가 다양해 인기가 많은 협업 도구입니다. 노션 서비스를 사용하기 위해 먼저 가입해 보겠습니다.

01 웹 브라우저에서 노션 홈페이지에 접속한 뒤, 화면 우측 상단에 있는 [Notion 무료로 사용하기] 버튼을 클릭하세요.

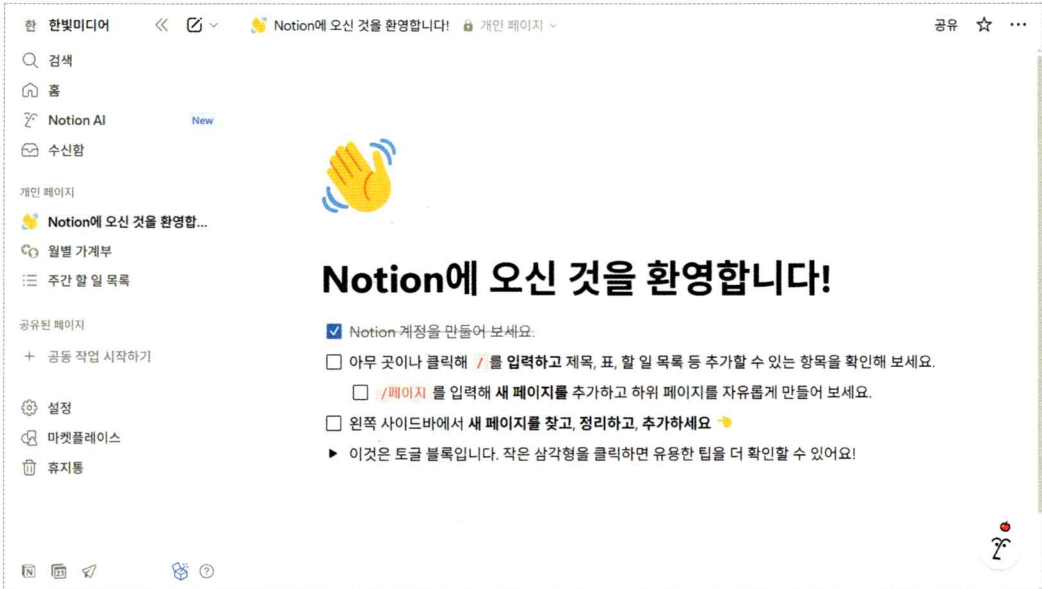
URL <https://www.notion.com>



02 계정이 있는 서비스와 연동하거나 새로 이메일 주소를 입력해 회원가입하세요.



03 계정 생성이 완료되면 워크스페이스를 생성할 수 있습니다. 이제 노션을 자유롭게 사용할 수 있습니다.



노션 MCP 서버 설치하기

클로드 코드에서 노션 MCP 서버를 설치해 연동해 봅시다. 우선 데스크톱의 'VibeCoding' 폴더에 새로운 'Study-06' 폴더를 만들고, 해당 위치에서 터미널을 실행해 준비합니다. 아직 클로드를 실행하지는 말고, 터미널을 열어둔 상태로 두세요.

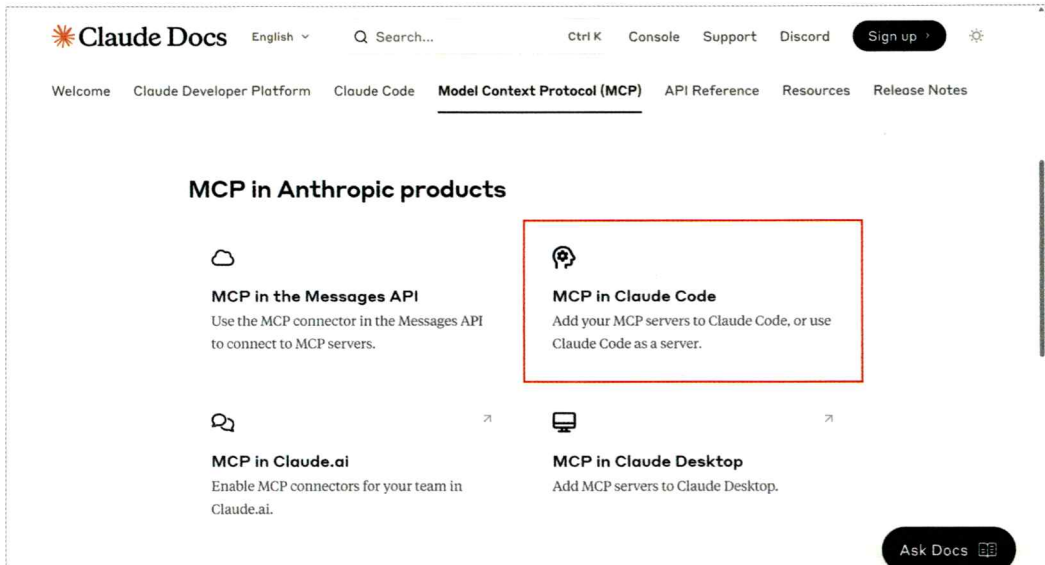
MCP 서버를 추가하는 과정은 세 단계로 정리할 수 있습니다.

- 1 터미널에서 `claude mcp add` 명령을 이용해 원하는 **MCP 서버를 설치**합니다.
- 2 노션처럼 외부 서비스와 연결되는 MCP의 경우에는 계정 연동을 위한 **인증** 과정을 진행합니다.
- 3 클로드 코드에서 해당 MCP의 기능을 실제로 사용할 수 있도록 **프롬프트를 작성**해 작업을 실행합니다.

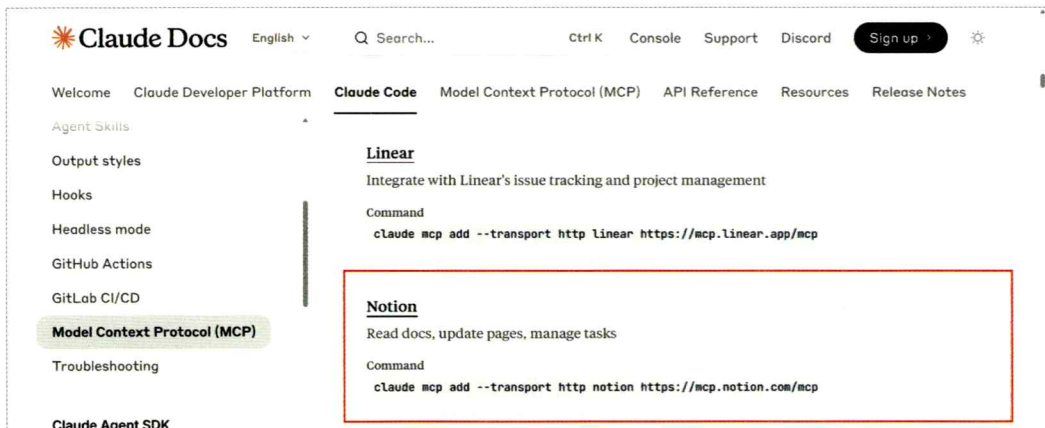
터미널에 설치 명령어를 입력해서 MCP 서버를 다운로드하고, 설정 파일에 등록할 것입니다. MCP 서버를 다운로드할 수 있는 명령어를 클로드 코드의 공식 문서에서 찾아봅시다.

01 먼저 웹 브라우저에서 클로드 코드의 MCP 공식 문서에 접속하세요. 그런 다음 화면을 내려 [MCP in Claude Code]를 찾아 클릭합니다.

URL <https://docs.claude.com/en/docs/mcp>



02 클로드 코드에 적용할 수 있는 MCP 서버 목록이 소개되어 있습니다. 화면을 스크롤해 'Project Management & Documentation' 목록에서 'Notion'을 찾으세요. 'Command' 항목에 있는 명령어 'claude mcp add --transport http notion https://mcp.notion.com/mcp'를 복사해 준비합니다.



note 깃허브 자료실의 [전체 프롬프트 모음 보기]에서도 MCP 설치 명령어를 찾을 수 있습니다.

03 실행한 터미널에 복사한 노션 MCP 서버의 설치 명령어를 입력합니다.

```
▶ claude mcp add --transport http notion https://mcp.notion.com/mcp
```

```
Microsoft Windows [Version 10.0.26100.6899]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user\Desktop\VibeCoding\Study-06>claude mcp add --transport
http notion https://mcp.notion.com/mcp
```

note 클로드 코드를 실행하기 전에 MCP 서버 설치 명령어를 먼저 입력하세요.

04 MCP 서버가 성공적으로 추가되었습니다.

```
Added HTTP MCP server notion with URL: https://mcp.notion.com/mcp to local
config
File modified: C:\Users\taehj\.claude.json [project: C:\Users\taehj\OneDrive\
Desktop\VibeCoding\Study-06]

C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-06>claude
```

MCP 서버 인증으로 연결 완성하기

01 MCP 서버를 설치했으니 터미널에 'claude' 명령어를 입력해 클로드 코드를 실행합니다.

```
C:\Users\user\Desktop\VibeCoding\Study-06>claude
Claude Code
Welcome back 훈공바이브!

Sonnet 4.5
Claude Pro
C:\Users\user\Desktop\VibeCoding\Study-06
```

02 클로드 코드 내 설치된 MCP의 목록과 서버 상태를 확인하기 위해 '/mcp' 명령어를 입력합니다.

```
▶ /mcp
```

```
> /mcp [reconnect <server-name>]

/mcp
  Manage MCP servers
```

03 MCP 서버 관리 창에서 노션 MCP가 등록되어 있는 것을 확인할 수 있습니다. 하지만 아직 로그인하지 않아 연결되지 않은 상태입니다. **[Enter]** 키를 눌러 연결을 진행해 봅시다.

화면 해설

MCP 서버 관리

1. 노션 △ 연결되어 있지 않음 • 로그인

MCP 설정 파일 위치:

- 사용자 설정 파일(모든 프로젝트에 공통으로 적용): C:\Users\user\.claude.json
- 프로젝트 설정 파일(특정 프로젝트에만 적용): C:\Users\user\Desktop\VibeCoding\Study-06\.mcp.json
- 로컬 설정 파일(현재 프로젝트에만 적용): C:\Users\user\.claude.json [project: C:\Users\user\Desktop\VibeCoding\Study-06]

MCP 서버 설정 파일 관련 도움말: <https://docs.claude.com/en/docs/claude-code/mcp>

```
> /mcp

Manage MCP servers

> 1. notion                      △ disconnected • Enter to login

MCP Config locations (by scope):
• User config (available in all your projects):
  • C:\Users\user\.claude.json
• Project config (shared via .mcp.json):
  • C:\Users\user\Desktop\VibeCoding\Study-06\.mcp.json (file does not exist)
• Local config (private to you in this project):
  • C:\Users\user\.claude.json [project: C:\Users\user\Desktop\VibeCoding\Study-06]

For help configuring MCP servers, see:
https://docs.claude.com/en/docs/claude-code/mcp
```

04 인증을 통해 클로드 코드가 노션 계정에 접근할 수 있게 해 봅시다. '1. Authenticate'를 선택하고 **[Enter]** 키를 누르세요.

화면 해설

노션 MCP 서버

상태: 인증이 필요합니다.

URL: <https://mcp.notion.com/mcp>

설정 정보 위치: C:\Users\user\.claude.json

C:\Users\user\Desktop\VibeCoding\Study-06

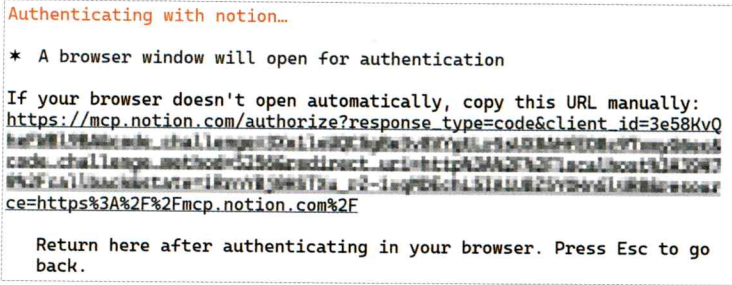
1. 인증하기

```
Notion MCP Server

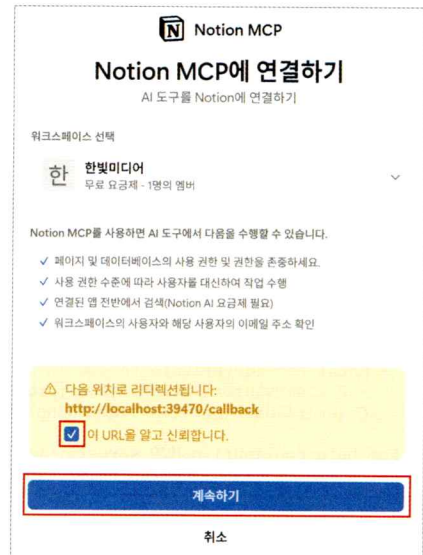
Status: △ needs authentication
URL: https://mcp.notion.com/mcp
Config location: C:\Users\user\.claude.json [project: C:\Users\user\Desktop\VibeCoding\Study-06]

> 1. Authenticate
   2. Disable
```

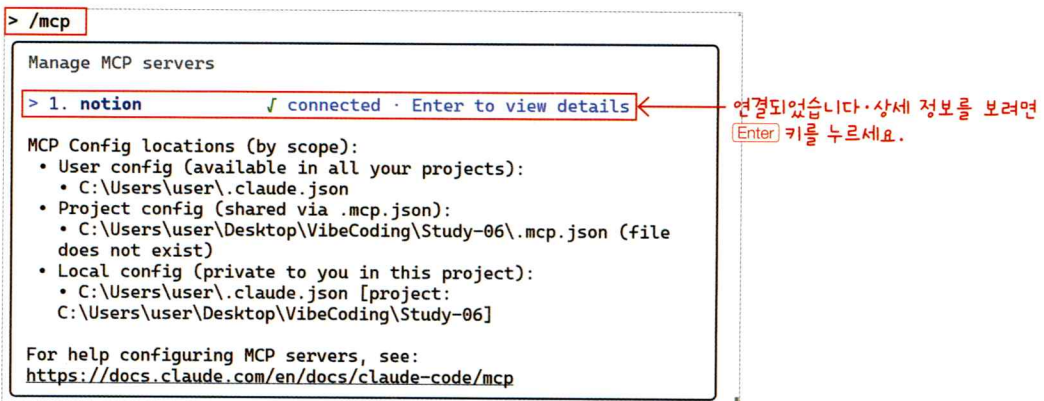
05 잠시 기다리면 웹 브라우저에 새 창이 열립니다.



06 MCP 서버에 노션 계정을 연결하기 위해 ‘이 URL을 알고 신뢰합니다.’ 체크박스를 클릭한 뒤, [계속하기] 버튼을 클릭합니다.



07 ‘Authentication Successful’이라는 메시지가 보이면 인증에 성공한 것입니다. 클로드 코드로 돌아가 다시 ‘/mcp’ 명령어를 입력해 보세요. 노션 MCP 서버가 연결되었음을 확인할 수 있습니다. **[Enter]** 키를 눌러 상세 정보를 확인해 봅시다.



08 노션 MCP 서버 안에는 클로드가 사용할 수 있는 15개의 작업 도구가 등록되어 있음을 확인할 수 있습니다. '1. View tools'를 선택한 뒤, **[Enter]** 키를 누르세요.

```
> /mcp

Notion MCP Server

Status: ✓ connected ✓ authenticated
URL: https://mcp.notion.com/mcp
Config location: C:\Users\user\.claude.json [project:
                C:\Users\user\Desktop\VibeCoding\Study-06]
Capabilities: tools · resources
Tools: 15 tools

> 1. View tools
   2. Re-authenticate
   3. Clear authentication
   4. Reconnect
   5. Disable

Esc to go back
```

09 노션 MCP 서버가 제공하는 구체적인 도구 목록을 볼 수 있습니다. **[Esc]** 키를 눌러 서버 관리 창에서 나가세요.

```
Tools for notion (15 tools)

> 1. notion-search      read-only, open-world
   2. notion-fetch       read-only
   3. notion-create-pages
   4. notion-update-page destructive
↓ 5. notion-move-pages

Esc to go back
```

노션 MCP로 자동 리서치 노트 만들기

이제 노션 MCP와 연동하여 실제 리서치 노트 자동화를 실행하겠습니다. 클로드 코드의 최신 업데이트를 검색하고 노션에 이를 요약해 업로드하게 해 보겠습니다.

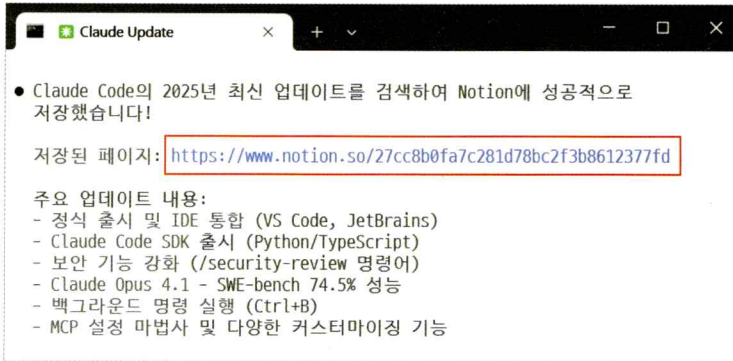
01 클로드 코드에 다음과 같이 프롬프트를 입력하세요.

프롬프트

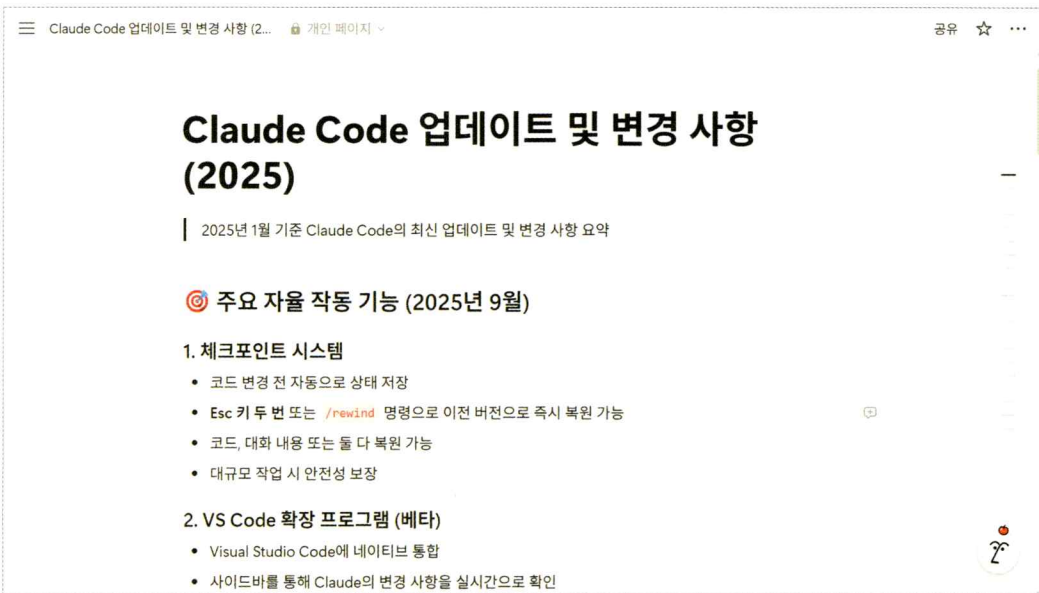
클로드 코드의 업데이트, 변경 사항을 검색하고 요약해 줘. 그 결과를 노션 MCP를 이용해 노션에 저장해 줘.

```
> 클로드 코드의 업데이트, 변경 사항을 검색하고 요약해 줘. 그 결과를
노션 MCP를 이용해 노션에 저장해 줘.■
```

02 '1. Yes'를 눌러 계속 진행하면 이와 같이 최신 업데이트를 검색해서 노션에 성공적으로 저장했다는 메시지를 볼 수 있습니다. 이제 실제로 노션에 문서가 생성되었는지 확인해 봅시다.



03 노션에 클로드 코드 최신 업데이트 문서가 생성되어 저장되어 있음을 볼 수 있습니다.



+ 여기서 잠깐 원하는 MCP 서버를 찾는 방법

MCP는 누구나 만들고 배포할 수 있기 때문에 현재에는 수천 가지가 넘는 MCP 서버들이 존재합니다. 클로드 코드의 MCP 공식 문서에도 몇 가지 MCP들이 있지만, 구글에서 원하는 MCP를 검색하면 더 많은 정보를 찾을 수 있습니다. 다양한 MCP 서버를 카테고리별로 분류해 비교 탐색할 수 있는 플랫폼도 많으니 참고하는 것도 좋습니다.

순차적 추론 MCP로 더 똑똑한 AI 만들기

클로드 코드에 MCP를 설치하고 실행하는 방법에 대해 조금 더 배워 보겠습니다.

로컬 MCP와 원격 MCP 이해하기

MCP 서버는 연결 방식에 따라 로컬^{local}과 원격^{remote} 두 가지로 나뉩니다. **로컬 MCP**는 MCP 중개 프로그램을 내 컴퓨터 안에서 직접 실행하는 방식이고, **원격 MCP**는 그 중개 프로그램마저 인터넷 상 외부 서버에서 실행하는 방식입니다. 두 방식 모두 MCP를 통해 클로드 코드가 외부 시스템과 연결되는 구조는 같지만, 연결 위치와 설정 과정이 다릅니다.

더 간단하게 시작할 수 있는 것은 원격 서버 방식입니다. 원격 MCP는 이미 외부 서버에 준비된 MCP에 접속해 사용하는 방식으로, 명령어 한 줄만 있으면 바로 연결할 수 있습니다.

예를 들어 노션 MCP를 설치할 때 사용했던 명령어 `claude mcp add --transport http notion https://mcp.notion.com/mcp`는 'notion'이라는 이름으로 이미 준비된 외부 서버 'https://mcp.notion.com/mcp'를 불러와 연결한다는 뜻입니다. 즉 클로드 코드가 해당 주소에 있는 MCP 서버에 접속해 노션과 직접 대화할 수 있게 되는 것입니다.

원격 MCP는 인터넷을 통해 접근 가능한 클라우드 서버에 연결되며, 대부분 로그인이나 인증 과정을 거쳐 사용자의 계정과 연결됩니다. 이 방식은 설정이 간편하고 여러 사용자와 공유하기 쉽다는 장점이 있지만, 반드시 인터넷이 연결된 환경이 필요하다는 점도 기억해야 합니다. 명령어의 구조는 다음과 같습니다.

```
▶ claude mcp add --transport http [내가 정한 이름] [서버 URL]
```

반면 로컬 MCP는 MCP를 내 컴퓨터 안에서 직접 실행해 사용하는 방식입니다. 이 방식은 인터넷 주소를 불러오는 대신 직접 실행할 명령어를 지정합니다.

로컬 MCP는 이미 준비된 URL을 입력하는 대신 실행할 명령어를 직접 지정해야 하기 때문에 약간의 설정이 필요하지만, 인터넷 없이도 작동하며 응답 속도도 더 빠르고 개인정보 보호에 유리합니다. 명령어의 구조는 다음과 같습니다. 여기서 '---' 기호는 클로드 코드의 옵션과 MCP 실행 명령을 구분하는 역할입니다.

```
▶ claude mcp add [내가 정한 이름] [옵션] -- [MCP 실행 명령]
```

원격과 로컬 MCP 중 어느 쪽을 선택할지는 상황에 따라 다릅니다. 빠르게 설치하고 여러 사람과 협업해야 한다면 원격, 네트워크가 불안정하거나 개인 환경에서 안전하게 작업하고 싶다면 로컬 MCP가 적합합니다. 많은 MCP 서버가 두 방식을 모두 지원하지는 않습니다. 둘 중 하나만 제공된다면 환경에 맞춰 선택하면 됩니다.

+ 여기서 잠깐 인증이란?

외부 서비스와 MCP를 연결할 때는, 접속하려는 사람이 정말 허가된 사용자인지 확인하기 위해 **인증** authentication이 필요합니다.

MCP 서버는 여러 사람의 데이터를 관리합니다. 누가 요청을 보내는지 모른다면, 다른 사람의 정보나 파일에 잘못 접근할 수 있겠죠. 따라서 인증을 통해 이 요청을 보낸 사람이 진짜 사용자임을 증명해야 합니다. 원격 MCP는 인터넷을 통해 통신하므로 중간에서 데이터가 유출될 위험이 있습니다. 인증 절차를 밟아 서버와 클라이언트가 암호화된 통신 하에 안전하게 데이터를 주고받을 수 있게 해야 합니다.

로컬 MCP 설치 명령어 이해하기

지금부터 로컬 MCP를 설치해 봅시다. 설치할 MCP는 **Sequential Thinking MCP**로, 엔트로픽이 만들어 배포한 공식 MCP 서버입니다. 이름 그대로 주어진 문제를 작은 단위로 분해해 순차적으로 추론하도록 돕는 도구입니다. 설치 명령어는 다음과 같습니다.

```
claude mcp add sequential-thinking -s local -- cmd /c npx -y  
               ①           ②           ③     ④           ⑤  
@modelcontextprotocol/server-sequential-thinking@latest  
               ⑥
```

이 한 줄의 명령 안에 MCP 설치에 필요한 모든 정보가 들어 있습니다. 각 부분을 차례로 살펴보겠습니다.

① sequential-thinking

클로드 코드에서 이 MCP를 구분하기 위한 이름입니다.

② -s local

MCP의 적용 범위를 지정합니다. 'local'은 현재 프로젝트에만 적용된다는 뜻입니다. project로 설정하면 같은 프로젝트 팀원들과 공유되고, user로 설정하면 내 계정 전체 프로젝트에서 사용할 수 있습니다. 기본값은 local이므로, -s 플래그를 생략해도 동일하게 적용됩니다.

-s 플래그는 --scope라는 옵션의 줄임형입니다. --scope를 사용해도 됩니다.

③ --

클로드 코드의 옵션과 인자를 나누는 기준입니다. 이 기호를 기준으로 앞부분에는 클로드 코드 옵션이, 뒷부분에는 MCP 서버 실행 명령이 위치합니다.

④ cmd /c

윈도우 환경에서 명령을 실행하고 종료하는 역할을 합니다. 맥OS나 리눅스 환경에서는 필요하지 않으므로 이 부분을 생략해도 됩니다.

⑤ npx -y

Node.js 패키지를 실행합니다. '-y'는 패키지가 로컬에 설치되어 있지 않을 때 자동으로 설치를 승인하도록 하는 옵션입니다. 즉, 'Ok to proceed? (y)' 메시지를 자동으로 건너뜁니다.

⑥ @modelcontextprotocol/server-sequential-thinking@latest

불러올 MCP 패키지명입니다. 패키지명은 '@조직명/패키지명@버전' 순서로 구성되어 있습니다. 즉, MCP 공식팀(modelcontextprotocol)이 배포한 Sequential Thinking MCP 패키지(server-sequential-thinking)의 최신 버전(latest)을 의미합니다.

개인이 만든 MCP 서버는 '@조직명'을 생략할 수 있습니다. '@버전' 부분도 생략할 수 있으며 '@2.0'과 같이 특정한 버전 번호를 붙이면 해당 버전을 사용하게 됩니다.

+ 여기서 잠깐 이중 하이픈 '--'의 두 가지 기능

명령어에서 짧은 옵션은 '-s'처럼 하이픈 한 개(-)에 한 글자를 붙여 쓰고, 긴 옵션은 '--scope'처럼 하이픈 두 개(--에 여러 글자를 붙여 씁니다.

이중 하이픈 '--' 뒤에 공백이 오면, 이것은 명령어의 옵션과 인자 사이를 구분하는 특별한 구분자가 됩니다.

```
claude mcp add mytool --scope local -- cmd /c ...
```

①

②

① --scope: 긴 옵션입니다. '--' 바로 뒤에 공백 없이 옵션 이름 'scope'가 붙어 있습니다.

② '--': 구분자입니다. '--' 뒤에 공백이 오고, 그 뒤에 'cmd /c ...'는 모두 인자로 처리됩니다.

순차적 추론 MCP 설치하고 활용하기

이제 실제로 Sequential Thinking MCP를 클로드 코드에 설치해 보겠습니다.

01 클로드 코드에 Sequential Thinking MCP 설치 명령어를 입력해 실행합니다.

```
▶ claude mcp add sequential-thinking -s local -- cmd /c npx -y  
@modelcontextprotocol/server-sequential-thinking@latest
```

```
Microsoft Windows [Version 10.0.26100.6899]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\user\Desktop\VibeCoding\Study-06>claude mcp add sequential-  
-thinking -s local -- cmd /c npx -y @modelcontextprotocol/server-se  
quential-thinking@latest
```

02 설치가 끝나면 'claude mcp list'를 입력해 클로드 코드에 설치된 MCP 목록을 확인합니다. 방
금 설치한 순차적 추론 MCP가 'sequential-thinking'이라는 이름으로 설치 완료되었습니다.

```
▶ claude mcp list
```

```
C:\Users\user\Desktop\VibeCoding\Study-06>claude mcp list  
Checking MCP server health...  
  
notion: https://mcp.notion.com/mcp (HTTP) - ✓ Connected  
sequential-thinking: cmd /c npx -y @modelcontextprotocol/server-seq  
quential-thinking@latest - ✓ Connected  
  
C:\Users\user\Desktop\VibeCoding\Study-06>
```

순차적 추론 MCP의 성능 확인하기

설치한 Sequential Thinking MCP의 추론 능력이 일반적인 추론과 얼마나 다른지 비교해 보겠습니
다. 목표는 웹 포트폴리오의 방문자가 사이트에 머무는 시간을 두 배로 늘리는 전략을 세우는 것입
니다.

이제 터미널에 'claude' 명령어를 입력해 클로드 코드를 실행합니다. 먼저 일반적인 방식으로 '방문
자 체류 시간을 늘리는 방법'을 요청해서 '체류 시간 늘리기'라는 제목으로 노선에 저장합니다. 이어
서 Sequential Thinking MCP를 이용해 같은 주제를 더 체계적이고 단계적인 방식으로 요청한 뒤
'체류 시간 늘리기 - 체계적인 구상'이라는 이름으로 노선에 저장해 비교해 볼 것입니다.

01 노션 MCP와 Sequential Thinking MCP를 사용해 목표를 달성하기 위한 방법을 구상하고 노션에 문서로 작성해 봅시다. 다음과 같이 프롬프트를 입력하세요.

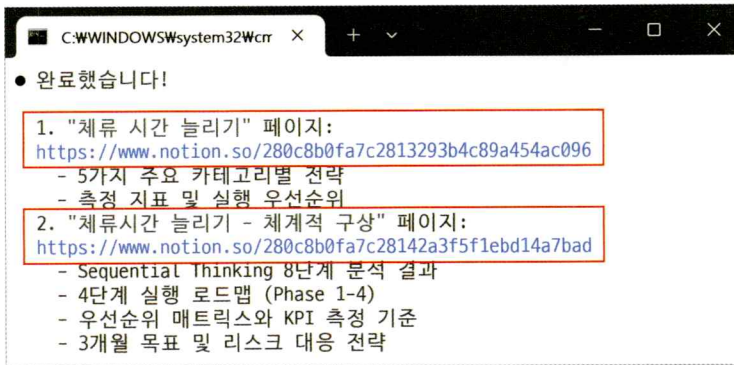
프롬프트

내 웹 포트폴리오 방문자의 체류 시간을 지금의 2배로 늘리고 싶어. 두 가지 문서를 작성해.

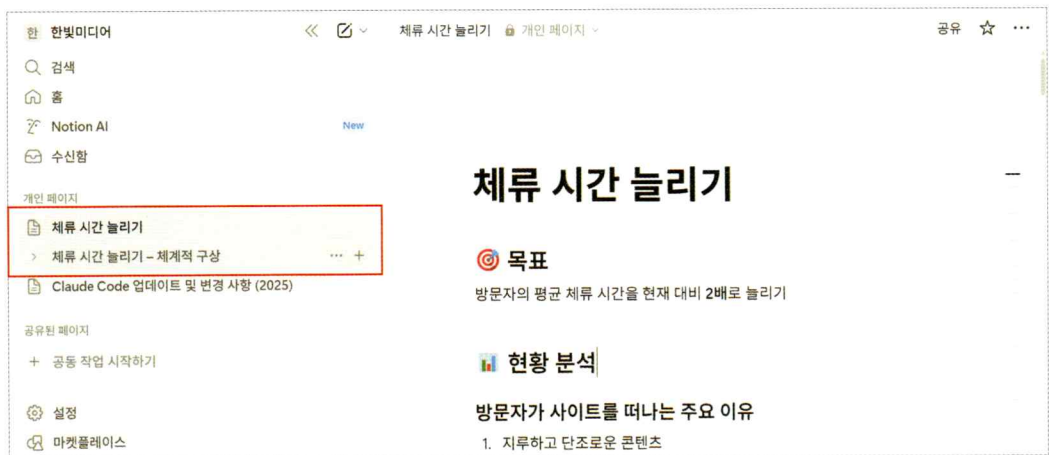
1. 이 목표를 달성할 방법을 구상해서 노션 MCP를 이용해 '체류 시간 늘리기'로 저장해 줘.
2. Sequential Thinking MCP를 이용해서 이 목표를 달성할 방법을 체계적으로 구상하고 노션 MCP를 이용해 '체류 시간 늘리기 - 체계적 구상'으로 저장해 줘.

> 내 웹 포트폴리오 방문자의 체류 시간을 지금의 2배로 늘리고 싶어. 두 가지 문서 작성해.
1. 이 목표를 달성할 방법을 구상해서 노션 MCP를 이용해 '체류 시간 늘리기'로 저장해 줘.
- 측정 지표 및 실행 우선순위
2. Sequential Thinking MCP를 이용해서 이 목표를 달성할 방법을 체계적으로 구상하고 노션 MCP를 이용해 '체류 시간 늘리기-체계적 구상'으로 저장해 줘. ■

02 잠시 기다리면 완료 메시지와 함께 두 개의 노션 문서 주소를 알려줍니다.



03 노션에 두 개의 새로운 페이지가 생성되었습니다.



04 두 문서의 크기를 비교해 봅시다. 일반 추론의 경우 총 85줄, 순차적 추론은 총 222줄로 분량에서 차이가 납니다. 또한 목차 역시 순차적 추론의 경우 목표, 실행 방법, 실행 로드맵, 측정 지표, 리스크 대응 등으로 더 상세합니다.

클라우드 코드	클라우드 코드 + 추론 MCP
일반 추론 필요 - 추론을 위한 추론 엔진 (예: PyTorch, TensorFlow) 1. 데이터 수집 및 전처리 - 데이터 수집 - 데이터 수집 스크립트 작성 - 데이터 수집 라이브러리 사용 (예: pandas, requests) - 데이터 전처리 - 데이터 정제 (결측치 처리, 중복 제거) - 데이터 분할 (학습 데이터, 검증 데이터, 테스트 데이터) - 데이터 스케일링 (정규화, 표준화) 2. 모델 학습 및 평가 - 모델 학습 - 모델 정의 (예: PyTorch, TensorFlow) - 모델 학습 (예: train.py) - 모델 평가 (예: validate.py) - 모델 평가 - 모델 성능 지표 (예: accuracy, loss) - 모델 성능 비교 (예: baseline vs. trained model) 3. 모델 배포 및 모니터링 - 모델 배포 - 모델 저장 (예: pickle, joblib) - 모델 배포 (예: Flask, FastAPI) - 모델 모니터링 - 모델 성능 모니터링 (예: Prometheus, Grafana) - 모델 성능 평가 (예: accuracy, loss)	순차적 추론 - Sequential Thinking MCP 분석 Sequential Thinking 분석 방법 - 추론 과정의 구조화 - 추론 과정의 흐름도 (Flowchart) 작성 - 추론 과정의 단계별 설명 (예: 데이터 수집, 전처리, 학습, 평가) - 추론 과정의 시각화 - 추론 과정의 흐름도 (Flowchart) 작성 - 추론 과정의 단계별 설명 (예: 데이터 수집, 전처리, 학습, 평가) 추론 과정의 구조화 - 추론 과정의 흐름도 (Flowchart) 작성 - 추론 과정의 단계별 설명 (예: 데이터 수집, 전처리, 학습, 평가) 추론 과정의 시각화 - 추론 과정의 흐름도 (Flowchart) 작성 - 추론 과정의 단계별 설명 (예: 데이터 수집, 전처리, 학습, 평가)

Sequential Thinking MCP는 프로젝트를 진행할 때 단계별 계획을 세워 체계적인 프롬프트나 PRD를 작성하는 데 유용하게 사용할 수 있습니다. 특히 여러 요소가 얽힌 복잡한 문제를 구조적으로 분석하고 정리해야 할 때 강력한 도구가 됩니다.

마무리

▶ 6가지 키워드로 정리하는 핵심 포인트

- **MCP**는 AI 모델이 외부 데이터, 도구, 서비스와 연결될 때 사용하는 표준화된 통신 규약입니다.
- **클라이언트 서버 모델**은 네트워크에서 정보를 요청하고 응답하는 구조를 말합니다. 클라이언트는 서버를 요청하는 쪽이고, 서버는 서비스를 제공하는 쪽입니다.
- **로컬 MCP**는 MCP 중개 프로그램을 내 컴퓨터 안에서 직접 실행하는 방식입니다.
- **원격 MCP**는 그 중개 프로그램마저 인터넷상 외부 서버에서 실행하는 방식입니다.
- **인증**은 접속하려는 사람이 정말 허가된 사용자인지 확인하는 절차입니다.
- 이중 하이픈 **--**은 두 글자 이상인 옵션 이름을 표시하거나, 명령어와 옵션을 구분할 때 쓰이는 기호로, 위치와 공백 사용에 따라 의미가 달라지므로 혼동에 주의해야 합니다.

▶ 확인 문제

1. MCP와 API의 차이점으로 가장 적절한 설명을 고르세요.

- ① API는 특정 기능을 요청하는 방식이고, MCP는 AI가 외부 시스템과 직접 소통하게 하는 프로토콜이다.
- ② MCP는 API보다 속도가 빠르다.
- ③ API는 유료이고 MCP는 무료이다.
- ④ MCP는 클로드 코드에서만 사용 가능하다.

2. 로컬 MCP 서버 설치 명령어에서 -- 구분자의 역할로 옳은 것을 고르세요.

- ① 주석을 표시하는 기호
- ② 클로드 코드 옵션과 MCP 서버 실행 명령을 구분
- ③ 버전 정보를 나타내는 표시
- ④ 에러 처리를 위한 구분자

3. Sequential Thinking MCP를 사용했을 때와 일반 추론의 차이점을 서술하세요.

08-2

MCP로 구현하는 완전 자동화 개발 환경

핵심 키워드

테스트

버전 관리

Context7

Playwright

원격 저장소

추가

커밋

푸시

이번 절에서는 Playwright MCP로 웹 애플리케이션을 자동으로 테스트하고, 깃허브 MCP로 프로젝트를 영구 보관하는 방법을 배워 봅니다. MCP 서버를 검색하고 설치하는 실무적인 방법부터 시작해, 실제로 쇼핑 리스트 앱을 만들고 자동 검증하는 과정까지 체계적으로 실습해 보겠습니다.

시작하기 전에

프로그램을 안정적으로 운영하려면 단순히 코드를 완성하는 것만으로는 충분하지 않습니다. 코드가 의도한 대로 실행되는지, 오류가 없는지를 확인하는 **테스트** test 과정이 필요합니다. 테스트를 통해 기능을 점검하고 새 코드를 추가해도 기존 기능이 손상되지 않게 할 수 있습니다. 특히 Playwright 같은 자동화 도구를 활용하면 반복적인 검증을 AI가 대신 수행해 개발자의 부담을 줄일 수 있습니다.

이처럼 테스트가 코드의 품질을 지켜준다면, **버전 관리** version control는 프로젝트의 안정성을 책임집니다. 코드의 모든 변화를 기록하고 필요할 때 이전 상태로 되돌릴 수 있어 실수나 오류로부터 안전하게 지켜주죠. 깃허브 GitHub를 이용하면 협업 과정에서도 각자의 변경 이력을 체계적으로 관리하며, 여러 사람이 동시에 작업해도 안정적으로 프로젝트를 유지할 수 있습니다.

원하는 MCP를 찾는 방법

MCP 서버를 찾는 가장 효과적인 방법은 **구글 검색**과 **깃허브 저장소**를 활용하는 것입니다. 대부분의 MCP 서버는 오픈소스로 공개되어 있어 깃허브에서 찾을 수 있으며, 공식 문서를 통해 설치 방법을 확인할 수 있습니다.

개발자들이 가장 많이 설치하는 두 개의 MCP를 설치해 보면서 그 과정을 알아보겠습니다.

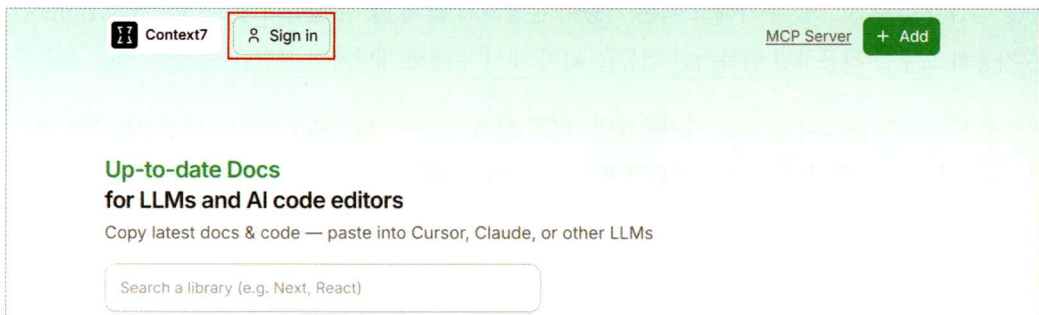
- **Context7**: 클로드 코드가 항상 최신 문서를 참고하도록 돕는 지식 연결 도구
- **Playwright**: 웹 사이트를 자동으로 테스트하고 UI를 검증하는 브라우저 제어 도구

Context7 API 키 발급받기

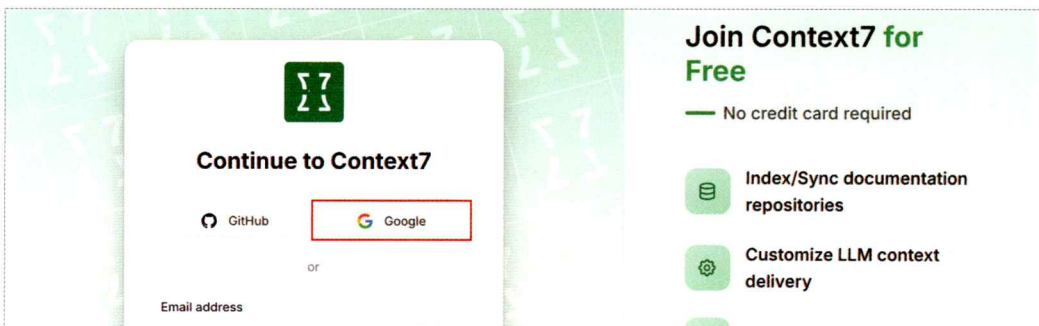
Context7은 AI 개발 툴이 간단한 참조 명령만으로 모든 코드를 최신 코드로 작성할 수 있게 해 줍니다. 먼저 Context7에 가입하고 API 키를 발급받은 뒤, MCP 설치 명령어를 찾아 클로드 코드에 설치하겠습니다.

01 먼저 Context7 홈페이지를 방문하고 [Sign in] 버튼을 눌러 회원가입을 시작합니다.

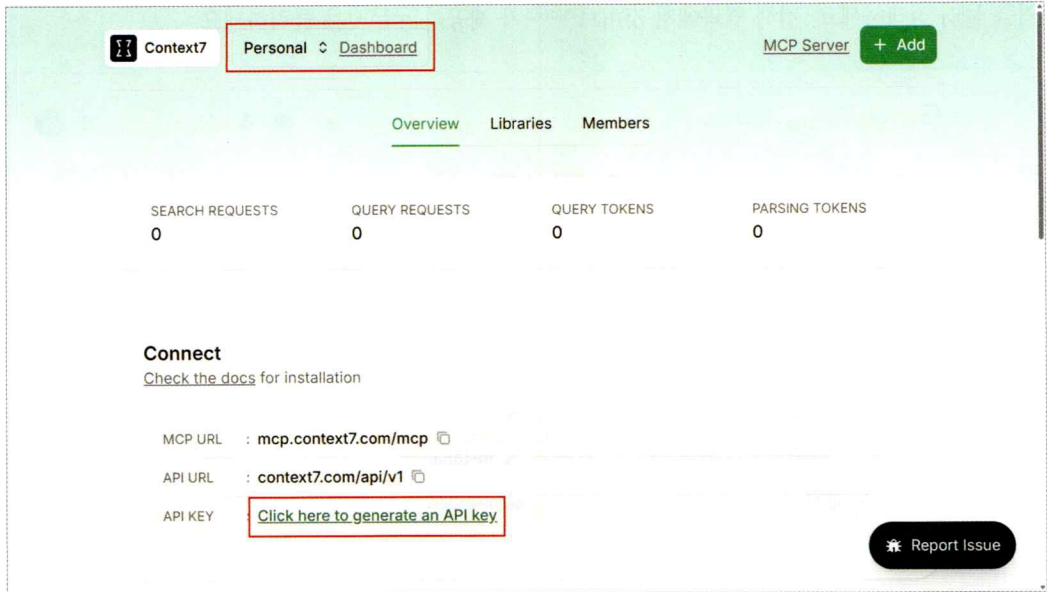
URL <https://www.context7.com>



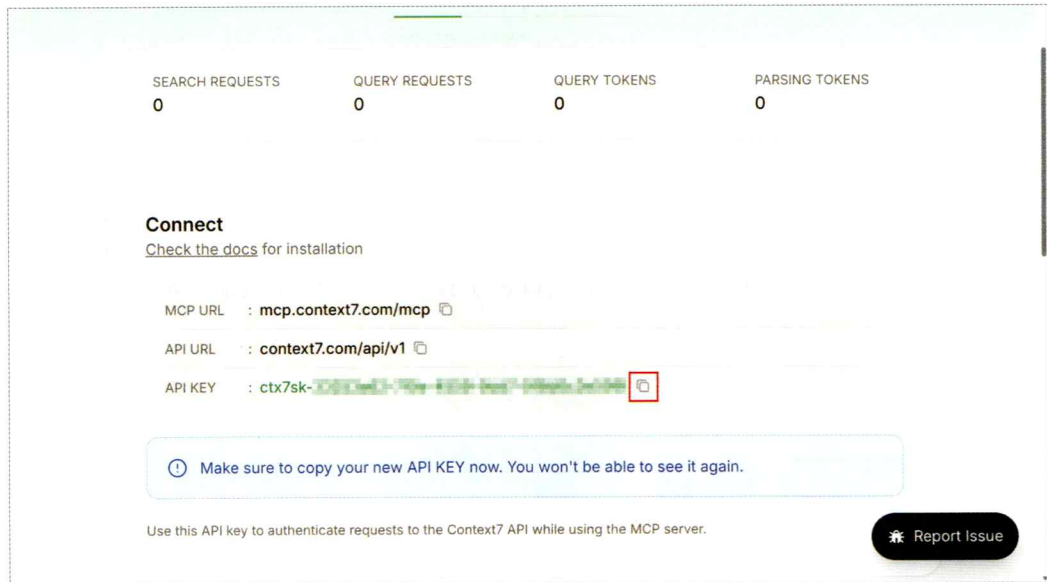
02 Context7이 제시하는 절차에 따라 가입하세요. 이 책에서는 구글 계정을 연동하겠습니다.



03 로그인하면 [Personal Dashboard] 화면으로 이동합니다. 'API KEY' 항목의 'Click here to go generate an API key'를 클릭해 Context7 API 키를 발급합니다.



04 '복사(📋)' 아이콘을 클릭해 API 키를 복사하고 보관해 두세요.

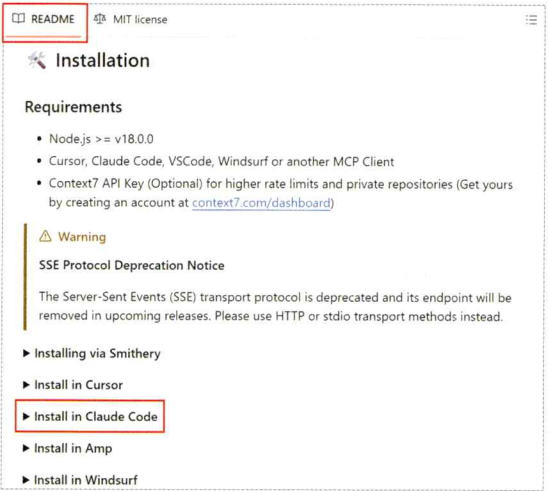


Context7 MCP 설치하기

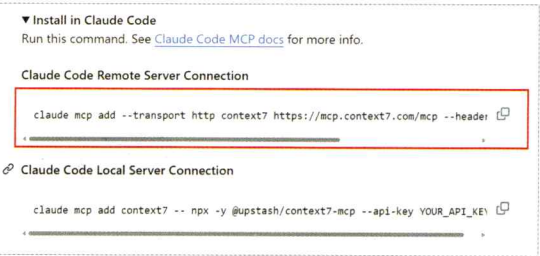
01 웹 브라우저에서 구글([google.com](https://www.google.com))에 접속한 뒤 검색창에 'Context7 mcp'을 입력하고 **Enter** 키를 눌러 검색하세요. 검색 결과에서 깃허브 [GitHub](https://github.com)가 제공하는 문서를 클릭하세요.



02 Context7 MCP에 대해 소개합니다. [README] 탭을 찾고 스크롤을 내리면 'Installation' 항목에서 설치 안내를 찾을 수 있습니다. [▶Install in Claude Code] 토글을 클릭하세요.



03 클로드 코드에 MCP를 설치할 수 있는 명령어가 원격과 로컬 두 가지 방식으로 제시됩니다. 이 책에서는 원격 MCP를 사용하기 위해 'Claude Code Remote Server Connection' 하위 명령어를 복사하겠습니다.



note 교재 깃허브 자료실의 [전체 프롬프트 모음 보기]에서도 MCP 설치 명령어를 찾을 수 있습니다.

04 데스크톱의 'Study-06' 폴더에서 터미널을 열고 복사한 Context7 MCP 설치 명령어를 입력하세요.

```
C:\Windows\System32\cmd
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-06>claude mcp add --transport http context7 https://mcp.context7.com/mcp --header "CONTEXT7_API_KEY: ctx7sk [redacted]"
```

05 설치가 완료되면 'claude mcp list'를 입력해 Context7 MCP가 잘 설치되었는지 확인하세요.

```
C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-06>claude mcp list
Checking MCP server health...

notion: https://mcp.notion.com/mcp (HTTP) - ✓ Connected
sequential-thinking: cmd /c npx -y @modelcontextprotocol/server-sequential-thinking@latest - ✓ Connected
context7: https://mcp.context7.com/mcp (HTTP) - ✓ Connected
```

Playwright MCP 설치하기

이번에는 웹 자동화 테스트 도구인 **Playwright**의 MCP를 찾아 설치해 보겠습니다. Playwright는 마이크로소프트 ^{Microsoft}가 개발한 오픈소스 프로젝트로, 웹 애플리케이션의 동작을 자동으로 테스트하는 도구입니다. 클로드 코드에 Playwright MCP를 연결해서 웹 페이지를 직접 열고 조작하며 테스트까지 실행하게 해 봅시다.

01 Playwright의 MCP 서버주소를 얻기 위해 구글의 검색창에 'Playwright MCP'를 입력하고, 검색 결과에서 깃허브 ^{GitHub}가 제공하는 문서를 클릭하세요.



02 [README] 탭의 'Getting started' 항목을 찾고, [Claude Code] 토글을 클릭하세요. Playwright MCP의 설치 명령어를 볼 수 있습니다.



잠시 멈춰서 명령어의 의미를 살펴봅시다.

claude mcp add

playwright

npx

@playwright/mcp@latest

1

2

3

4

- 1 클로드 코드에 새로운 MCP 서버를 추가하는 명령어
- 2 설치할 MCP의 이름
- 3 Node.js 환경에서 패키지를 바로 실행할 수 있게 해주는 명령어
- 4 Playwright팀이 배포한 MCP 패키지 이름(playwright/mcp)과 버전(@latest)

이 설치 명령어에 앞서 공부한 몇 가지 옵션을 추가해서 프로젝트 환경에 맞게 Playwright를 설치해 봅시다. 추가할 옵션은 다음과 같습니다.

옵션	의미	설명
-s local	범위 지정 옵션	현재 프로젝트 폴더에만 적용된다는 뜻
--	구분자	명령어와 옵션 사이의 구분자. 공백 포함 필수
cmd /c	윈도우 명령어 실행 옵션	윈도우 환경에서 명령어를 실행하라는 뜻

note 맥OS, 리눅스에서는 'cmd/c' 옵션을 생략할 수 있습니다.

03 이제 클로드 코드에 Playwright MCP를 설치해 봅시다. 'Study-06' 폴더에서 터미널을 실행한 뒤 다음과 같이 명령어를 입력하세요.

▶

claude mcp add playwright

-s local

-- cmd /c

npx @playwright/mcp@latest

↑

추가한 옵션

```
Microsoft Windows [Version 10.0.26100.6899]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user\Desktop\VibeCoding\Study-06>claude mcp add playwright
-s local -- cmd /c npx @playwright/mcp@latest
```

04 설치가 완료되면 'claude mcp list' 명령어를 통해 확인합니다.

```
▶ claude mcp list
```

```
C:\Users\user\Desktop\VibeCoding\Study-06>claude mcp list
Checking MCP server health...

notion: https://mcp.notion.com/mcp (HTTP) - ✓ Connected
sequential-thinking: cmd /c npx -y @modelcontextprotocol/server-seq
quential-thinking@latest - ✓ Connected
playwright: cmd /c npx @playwright/mcp@latest - ✓ Connected
```

쇼핑 리스트 애플리케이션 만들고 MCP로 테스트 자동화하기

웹 개발에서 가장 중요한 작업이 **테스트**입니다. 모든 버튼이 제대로 작동하는지, 입력 필드가 올바르게 검증되는지, 데이터가 정확히 저장되는지를 매번 수동으로 확인하는 일은 시간이 많이 걸리고 실수하기 쉽습니다. Playwright MCP를 사용하면 이런 반복적인 테스트 과정을 AI에게 맡겨 자동으로 수행할 수 있습니다.

이번 실습에서는 간단한 쇼핑 리스트 애플리케이션을 만들어 봅시다. 쇼핑 리스트에 상품을 추가하거나 삭제하고 구매한 상품에는 체크해서 삭제하는 기능이 있는 간단한 웹 애플리케이션입니다.

쇼핑 리스트 앱 만들기

01 터미널에서 클로드 코드를 실행하고 다음과 같이 프롬프트를 입력하세요.

프롬프트

쇼핑 리스트 앱을 만들어 줘. 아이템 추가, 삭제, 체크 기능이 있는 간단한 웹 UI로 로컬 브라우저에서 실행되게 해 줘.

```
C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-06>claude

Claude Code v2.0.9
Sonnet 4.5 · Claude API
C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-06
```

```
> 쇼핑 리스트 앱을 만들어 줘. 아이템 추가, 삭제, 체크 기능이 있는 간단한
  웹 UI로 로컬 브라우저에서 실행되게 해 줘. █
```

Thinking off (tab to toggle)

02 작업이 완료되면 'Study-06' 폴더에 생성된 'shopping-list.html' 파일을 더블 클릭하여 실행합니다.

● 쇼핑 리스트 앱을 만들었습니다! `shopping-list.html` 파일을 브라우저에서 열면 바로 실행됩니다.

주요 기능:

- ☒ 아이템 추가 (입력 후 "추가" 버튼 또는 Enter 키)
- ☒ 체크박스로 구매 완료 표시
- ☒ 삭제 버튼으로 아이템 제거
- ☒ 로컬 스토리지에 자동 저장 (브라우저를 달아도 데이터 유지)
- ☒ 반응형 디자인 및 애니메이션 효과

파일을 더블클릭하거나 브라우저로 드래그하여 실행하세요!

note 파일의 이름은 책과 다를 수 있습니다.

03 '쇼핑 리스트' 애플리케이션이 완성되었습니다.



Playwright MCP로 자동 테스트 실행하기

이제 Playwright MCP를 실행해 이 앱이 제대로 작동하는지, 자동 테스트를 실행해 보겠습니다. '쇼핑 리스트 앱'의 주요 기능은 ❶ 아이템 추가, ❷ 체크 박스로 구매 완료 표시, ❸ 삭제 버튼으로 아이템 제거입니다. 클로드 코드가 애플리케이션에 직접 접근해 이 세 가지 기능을 테스트할 것입니다.

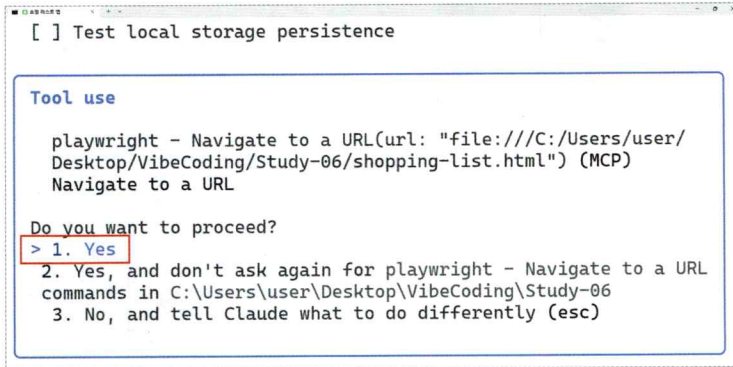
01 클로드 코드에 다음과 같이 입력해서 애플리케이션 테스트를 요청합니다.

프롬프트

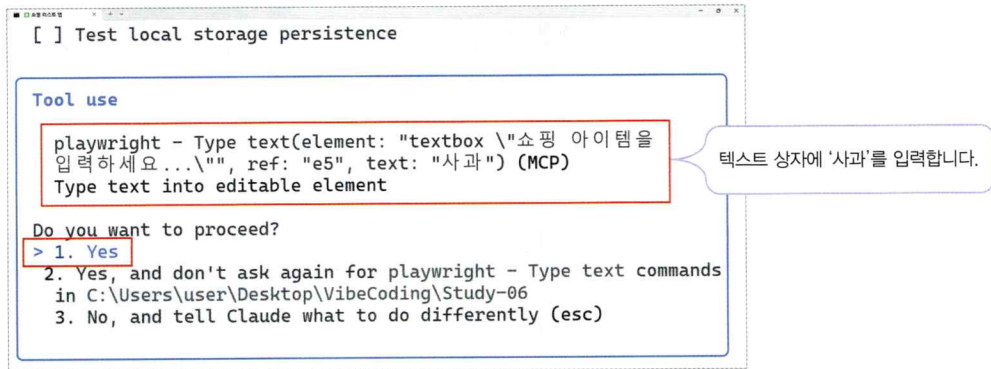
Playwright MCP를 사용해서 이 쇼핑 리스트 앱의 모든 기능을 자동으로 테스트해 줘. 아이템 추가, 삭제, 체크 기능이 제대로 작동하는지 확인해 줘.

> Playwright MCP를 사용해서 이 쇼핑 리스트 앱의 모든 기능을 자동으로 테스트해 줘.
아이템 추가, 삭제, 체크 기능이 제대로 작동하는지 확인해 줘.■

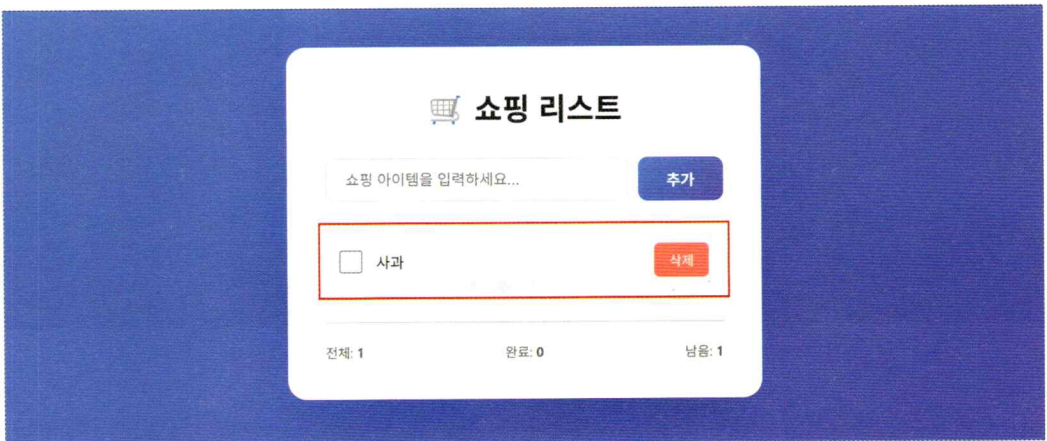
02 그러면 클로드 코드가 해당 MCP를 통해 HTML 파일을 열고 테스트를 시작하는 것에 대한 승인 요청이 나타납니다. '1. Yes'를 눌러 계속 진행합니다.



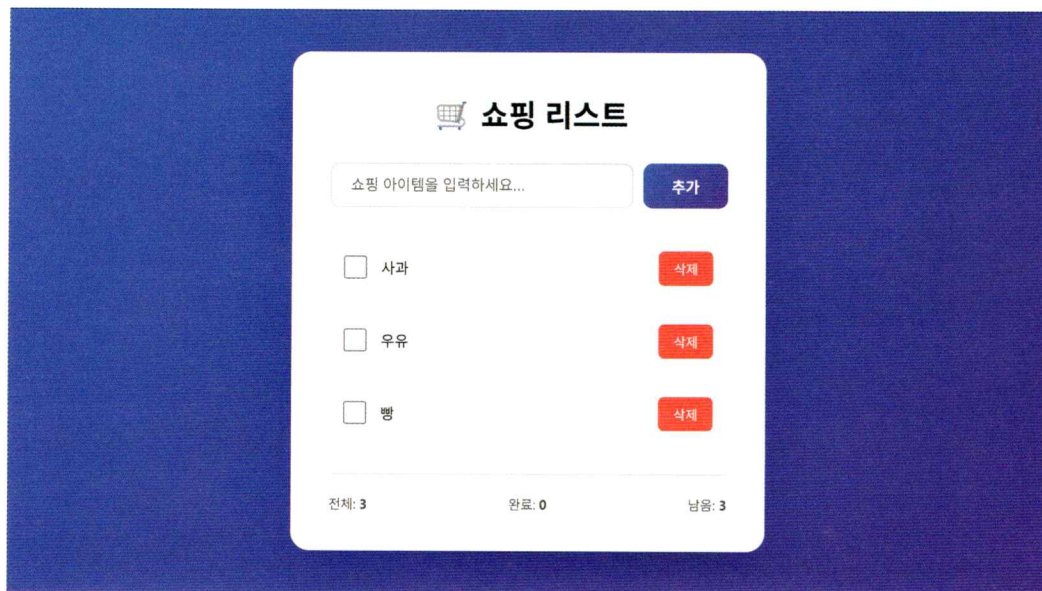
03 클로드 코드가 아이템 추가 기능이 잘 작동하는지 테스트하기 위해, 입력창에 '사과'라는 텍스트를 자동으로 입력하려고 합니다. '1. Yes'를 선택해 실행합니다.



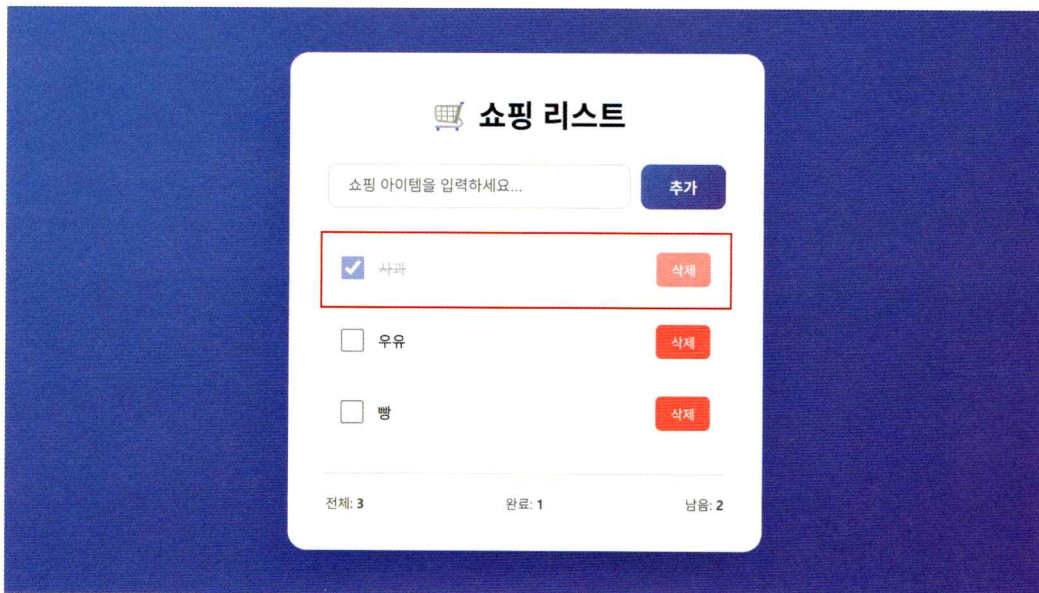
04 브라우저가 자동으로 열리면서 쇼핑 리스트에 '사과'가 추가되었습니다.



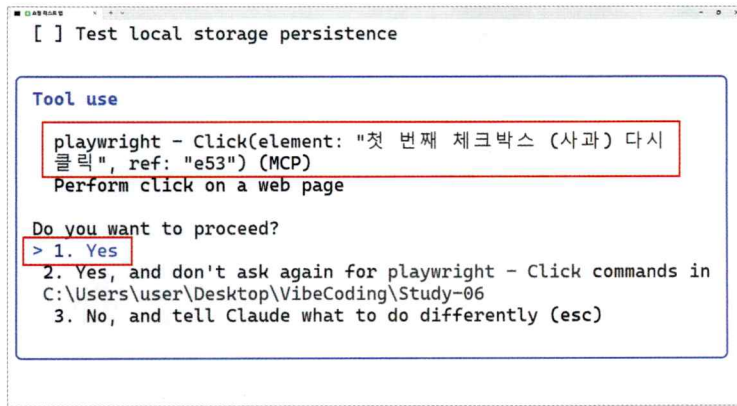
05 클로드 코드의 Playwright MCP가 계속해서 '우유', '빵' 등 아이템을 추가해, 리스트가 제대로 작동하는지 테스트합니다. 이로써 아이템 추가 기능 테스트를 성공적으로 마쳤습니다.



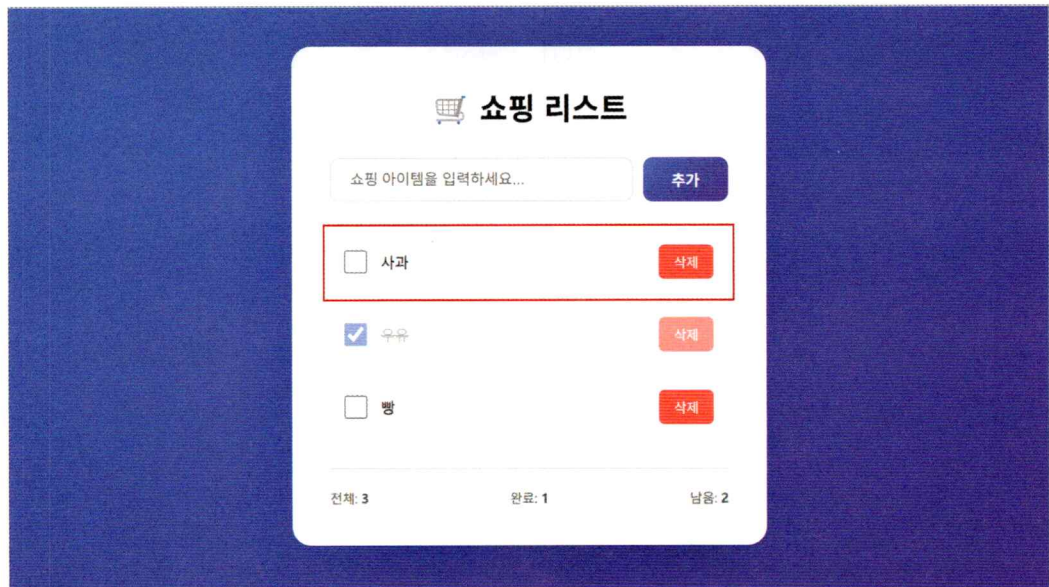
06 그다음은 구매한 아이템의 체크 박스를 클릭해서 삭선하는 기능을 점검합니다. 자동으로 체크 박스가 클릭되며 '사과' 아이템이 삭선되는 것을 확인할 수 있습니다.



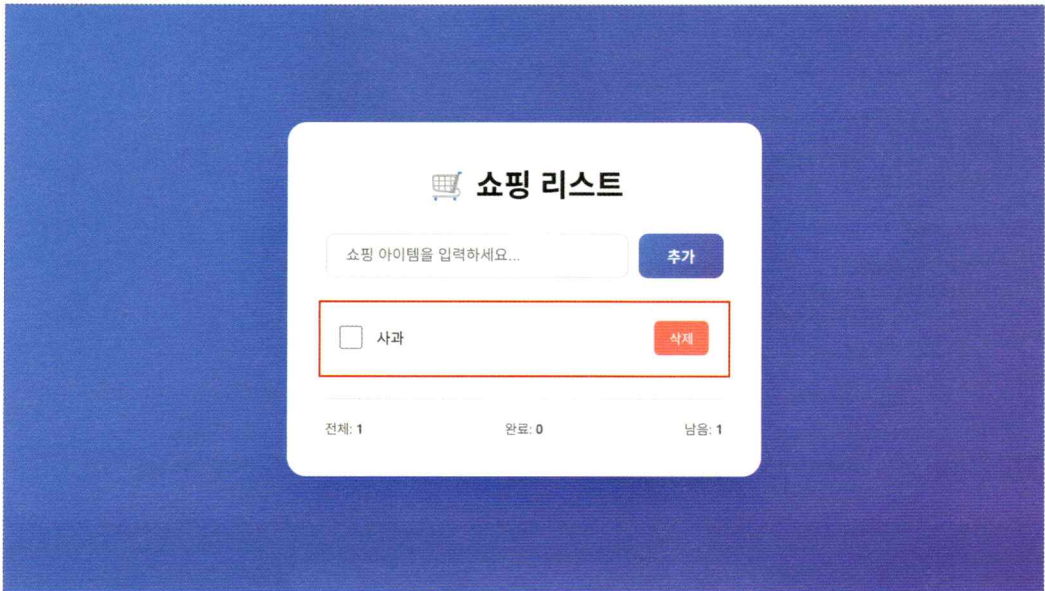
07 삭선을 해제할 것인지 묻고 있습니다. '1.Yes'를 선택해 체크 박스를 다시 클릭해서 삭선이 해제되는지 점검합니다.



08 정상적으로 해제되네요. 이로써 체크 박스 기능이 잘 작동하는지도 테스트했습니다.



09 마지막으로 아이템 삭제 기능이 제대로 작동하는지 확인합니다. 쇼핑 리스트에서 '우유'와 '빵'이 자동으로 삭제됩니다.



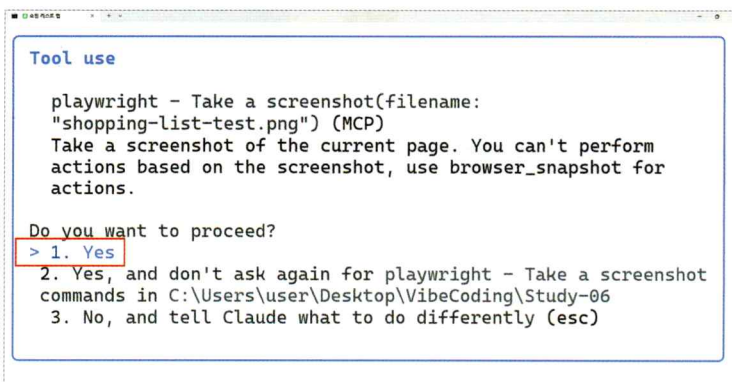
10 Playwright MCP가 진행한 테스트를 기록하기 위해, 모든 테스트가 완료되면 현재의 화면을 스크린샷으로 남길지 묻습니다. '1. Yes'를 눌러 진행시킵니다.

화면 해설

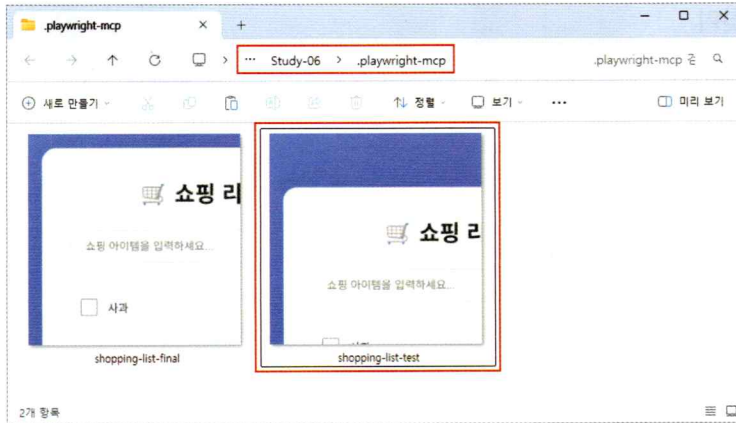
playwright - 스크린샷 촬영(파일명: shopping-list-test.png)

현재 페이지의 스크린샷을 촬영합니다.

스크린샷을 기반으로 동작을 수행할 수는 없습니다. 액션을 수행하려면 browser_snapshot 명령을 사용하세요.



11 'Study-06' 폴더에 테스트 폴더 '.playwright-mcp'가 생성되고, 폴더 안에는 'shopping-list-test.png' 파일이 저장됩니다. 이 기능은 테스트 결과를 기록하고 실제 화면을 보고할 때 유용하게 쓸 수 있습니다.



12 모든 테스트가 정상적으로 진행되었음이 보고됩니다.

- ✔ 쇼핑 리스트 앱 자동 테스트 완료!

Playwright MCP를 사용하여 모든 기능을 테스트했습니다. 결과 요약:

테스트 시나리오

 - 사과, 우유, 빵 추가 → 전체 3개
 - 사과, 우유 체크 → 완료 2개, 남음 1개
 - 사과 체크 해제 → 완료 1개, 남음 2개
 - 빵, 우유 삭제 → 전체 1개 남음
 - 계란, 치즈 추가 → 전체 3개
 - 계란 체크 → 완료 1개, 남음 2개
 - 페이지 새로고침 → 모든 데이터 유지 확인 ✔

결론

❌ 모든 기능이 완벽하게 작동합니다! 앱이 프로덕션 준비 상태입니다.

깃허브 MCP로 프로젝트 영구 보관하기

깃허브 GitHub는 전 세계 개발자가 사용하는 코드 **버전 관리** 플랫폼입니다. 클로드 코드의 MCP 서버를 이용하면 로컬 환경에서 작성한 웹 애플리케이션을 깃허브에 자동으로 업로드할 수 있습니다.

일반적으로 코드를 수정할 때마다 직접 깃 명령어를 입력해 변경 내용을 업로드해야 하지만, 깃허브 MCP 서버를 연결하면 클로드 코드가 이 과정을 대신 처리합니다. AI가 작성한 코드를 자동으로 깃허브 저장소에 업로드하고, 버전을 관리하는 시스템을 구현할 수 있는 것이죠.

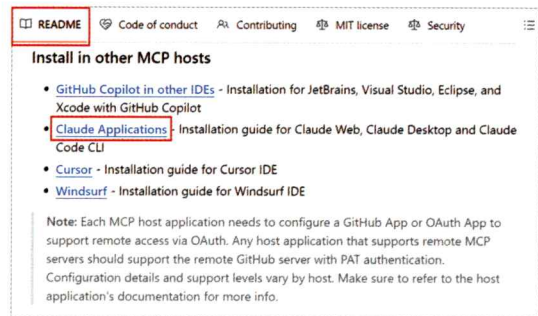
그러려면 깃허브 MCP 서버 주소가 필요합니다. 지금부터 ‘쇼핑 리스트’ 애플리케이션을 깃허브에 업로드해 봅시다.

깃허브 MCP 설치 명령어 찾기

01 구글에 ‘GitHub MCP’를 검색한 뒤, 검색 결과 중 깃허브가 제공하는 문서를 클릭합니다.



02 [READ ME] 탭의 ‘Install in other MCP hosts’ 항목에서 ‘Claude Applications’를 클릭하세요.



03 클로드 코드 CLI에서 원격 MCP 서버를 설치할 수 있는 명령어가 있습니다.

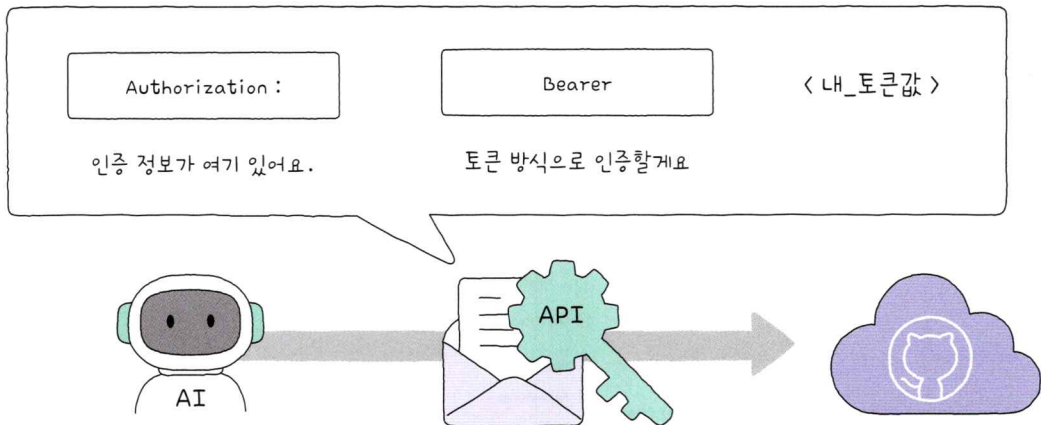


명령어는 다음과 같습니다.

```
▶ claude mcp add --transport http github https://api.githubcopilot.com/mcp -H  
"Authorization: Bearer $(grep GITHUB_PAT .env | cut -d '=' -f2)"
```

깃허브는 사용자 인증을 위해 **API 토큰**^{token} 방식을 사용합니다. 따라서 깃허브 MCP 서버 설치 명령어에는 사용자 인증과 관련한 옵션이 포함되어 있습니다.

- **-H**는 '--header'의 축약형으로, MCP 서버에 사용자 인증 데이터를 전달하기 위해 HTTP 요청에 헤더(header) 정보를 추가하라는 옵션입니다.
- **Authorization: Bearer**가 깃허브가 인증을 위해 API에 접근할 때 사용하는 표준 형식입니다. 'Authorization'은 인증 정보를 전달한다는 뜻이고, 'Bearer'는 인증 방식이 토큰 형식이라는 의미입니다.
- **\$(grep GITHUB_PAT .env | ...)**는 API 키가 들어갈 자리입니다. '.env' 파일에서 'GITHUB_PAT' 변수를 포함한 줄을 찾아 명령어에 치환하라는 의미입니다.

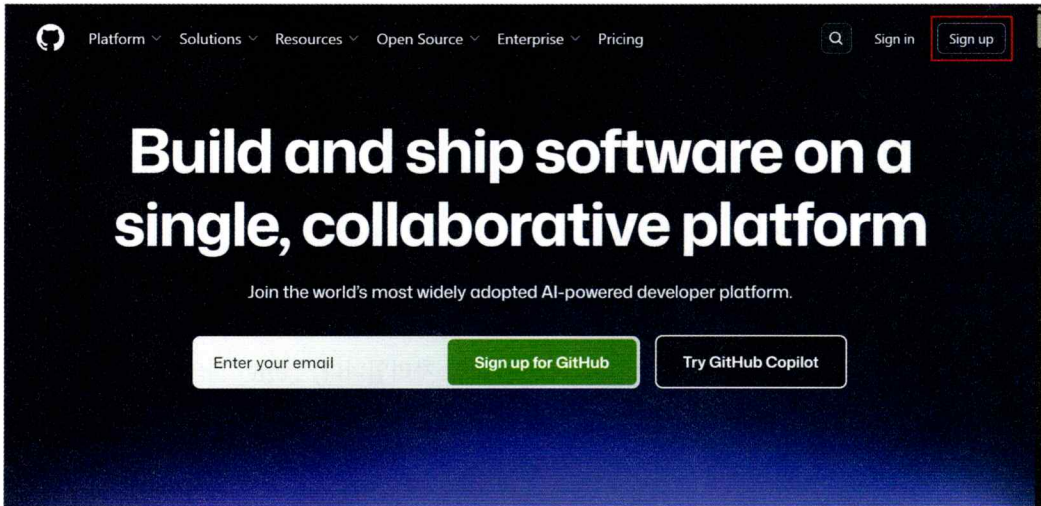


깃허브 MCP 서버를 클로드 코드에 설치하기 위해서는 API 토큰이 필요하다는 것을 알았습니다. 이제 깃허브에서 API 토큰을 발급받아 명령어에 대치해 보겠습니다.

깃허브 API 토큰 발급받기

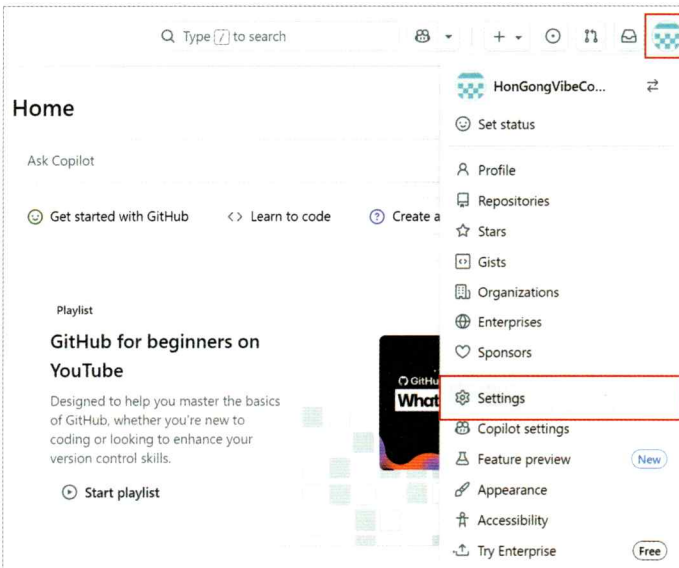
01 API 토큰을 얻기 위해 깃허브 홈페이지에 로그인하겠습니다. [Sign up] 버튼을 클릭해 회원가입을 진행하세요.

URL <https://www.github.com>

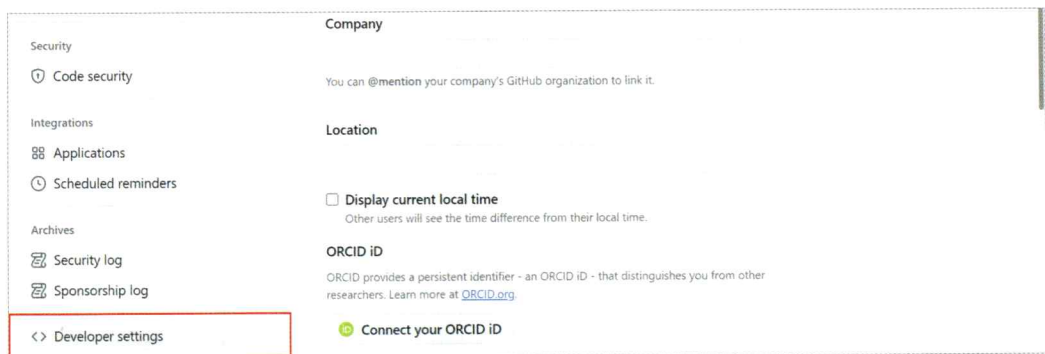


note 이미 깃허브 계정을 가지고 있다면 [Sign in] 버튼을 눌러 로그인을 진행하면 됩니다.

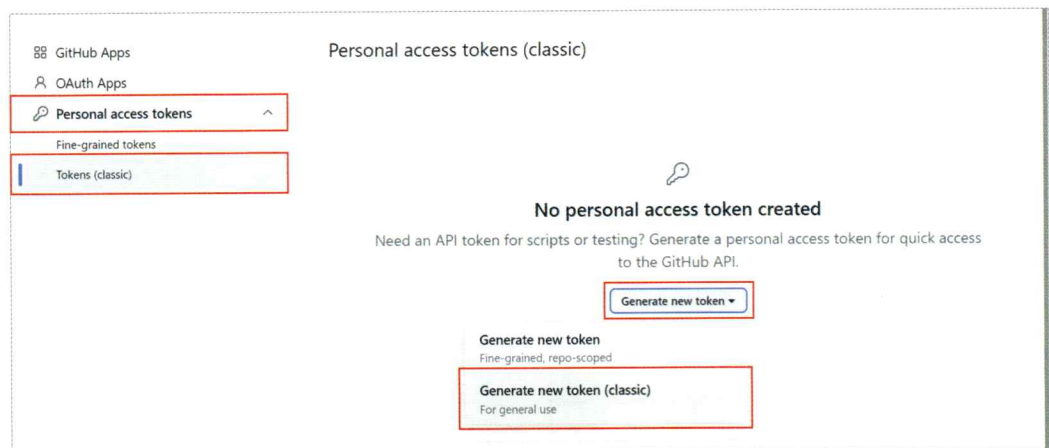
02 깃허브에 로그인한 뒤, 오른쪽 상단의 프로필을 클릭하고 [Settings]를 클릭합니다.



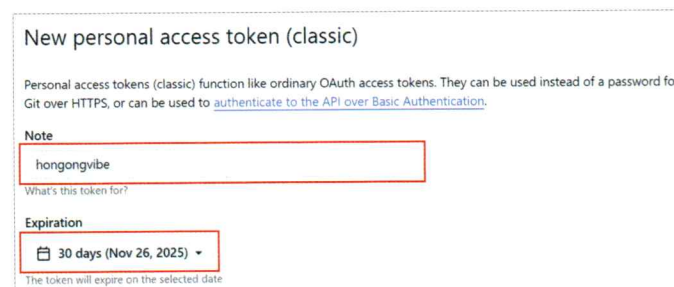
03 왼쪽 사이드바에서 스크롤을 내려 [Developer settings] 메뉴를 클릭합니다.



04 왼쪽 사이드바의 [GitHub Apps] 메뉴에서 [Personal access tokens] - [Tokens (classic)] 을 차례로 클릭한 뒤, 화면 중앙에 [Generate new token] 드롭다운에서 [Generate new token (classic)]을 클릭합니다.



05 새 토큰을 만들 수 있습니다. 'Note' 입력란에 'hongongvibe'라고 입력하고, [Expiration(유효 기간)]은 '30 days'로 지정합니다.



06 ‘Select scopes’는 개인 액세스 토큰이 접근할 수 있는 권한 범위를 정의하는 옵션입니다. 다음 세 가지 옵션을 선택한 뒤, [Generate Token] 버튼을 클릭하세요.

- repo: 개인 저장소 전체 제어 권한
- user: 사용자 데이터 전체 업데이트 권한
- project: 프로젝트 전체 제어 권한

07 API 토큰이 발급되었습니다. ‘복사(📋)’ 아이콘을 클릭하고 API 토큰을 저장해 두세요.

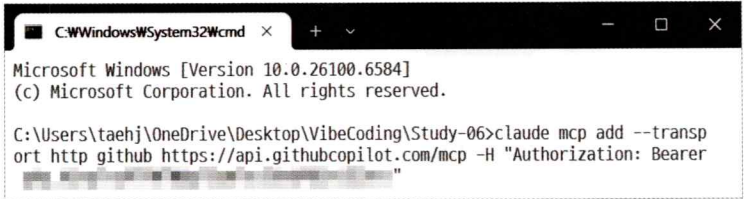
note 다른 API와 마찬가지로 깃허브 API 토큰 역시 단 한 번만 보여집니다. 반드시 별도의 저장소에 복사해 두는 것을 잊지 맙시다.

깃허브 MCP 설치 완료하기

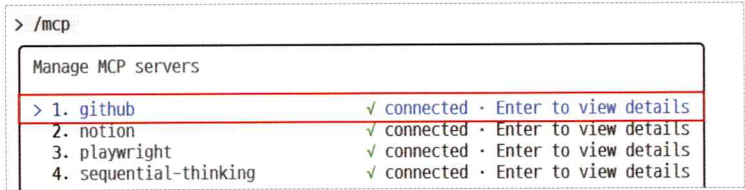
발급한 깃허브 API 토큰을 사용해서 클로드 코드에 깃허브 MCP 서버를 설치하겠습니다. 터미널을 실행하고, 앞서 깃허브 MCP 설치 명령어를 준비하세요.

01 터미널에 다음과 같이 깃허브 MCP 설치 명령어를 입력합니다. 이때 <깃허브 API 토큰> 위치에 깃허브 API 토큰으로 대체합니다.

```
▶ claude mcp add --transport http github https://api.githubcopilot.com/mcp -H "Authorization: Bearer <깃허브 API 키>"
```



02 깃허브 MCP 서버 설치를 마치고 안내가 나오면 ‘claude’ 명령어를 입력해 클로드 코드를 실행합니다. 그리고 나서 ‘/mcp’ 명령어를 통해 깃허브 MCP가 무사히 설치되었고 ‘connected’로 표시되는 것을 확인합니다.

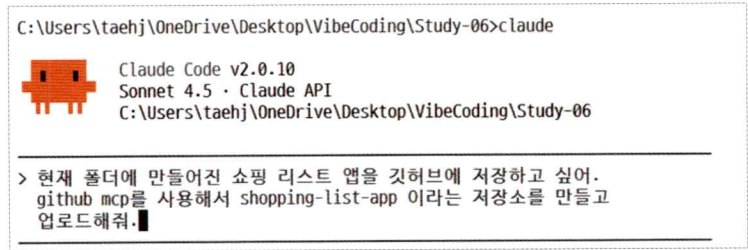


내 프로젝트 업로드하기

깃허브 MCP가 설치되면 이제 작업한 ‘쇼핑 리스트’ 앱을 깃허브 저장소를 만들어 업로드해 보겠습니다. ‘Study-06’ 폴더 위치에서 클로드 코드를 실행하세요.

01 다음과 같이 프롬프트를 입력합니다.

```
프롬프트  
현재 폴더에 만들어진 쇼핑 리스트 앱을 깃허브에 저장하고 싶어. github mcp를 사용해서 shopping-list-app이라는 저장소를 만들고 업로드해 줘.
```



여기서 잠깐 깃허브와 Git 명령어 이해하기

깃허브는 코드를 온라인에 저장하고 관리하는 클라우드 버전 관리 플랫폼입니다. 여러 사람과 함께 작업할 수도 있고, 코드의 변경 이력을 기록해서 언제든지 이전 버전으로 되돌릴 수도 있습니다. 깃허브에서의 작업 흐름은 ① **추가** add, ② **커밋** commit, ③ **푸시** push 순으로 이루어집니다.



① 추가

깃허브에 프로젝트를 저장하기 위해서 먼저 깃허브 계정에 코드를 저장할 수 있는 **원격 저장소** remote repository를 만듭니다. 그리고 로드할 파일들을 선택해서 **준비 공간** staging area에 추가합니다.

② 커밋

준비 공간에 추가된 파일들이 확정되면 이를 전송 바로 전 단계인 **로컬 저장소**에 보관합니다. 이 과정을 커밋이라고 하는데, 변경 내용을 저장하는 단위를 의미하기도 합니다. 즉 커밋은 코드의 현재 상태를 로컬 저장소에 저장하는 작업입니다.

③ 푸시

로컬 저장소에 커밋된 파일을 원격 저장소로 업로드하는 과정을 푸시라고 합니다. 코드를 푸시하면 팀원들이 깃허브에서 변경 내역을 확인할 수 있습니다.

클라우드 코드의 GitHub MCP는 이러한 과정을 자동으로 처리해서 더욱 편리하게 깃허브를 사용할 수 있게 해줍니다.

02 클로드 코드가 저장소를 생성하고, 추가 add와 커밋 commit 등의 명령을 실행하기 위한 승인을 요청합니다. '1. Yes'를 눌러 푸시 push까지 계속 진행시킵니다.

GitHub Upload
+
-
□
×

```
}

• 저장소가 생성되었습니다! 이제 Git 설정 및 파일을 업로드하겠습니다.
• Bash(git init && git add shopping-list.html CLAUDE.md && git commit -m
  "Initial commit: Shopping list web application")
  L Running...
```

Bash command

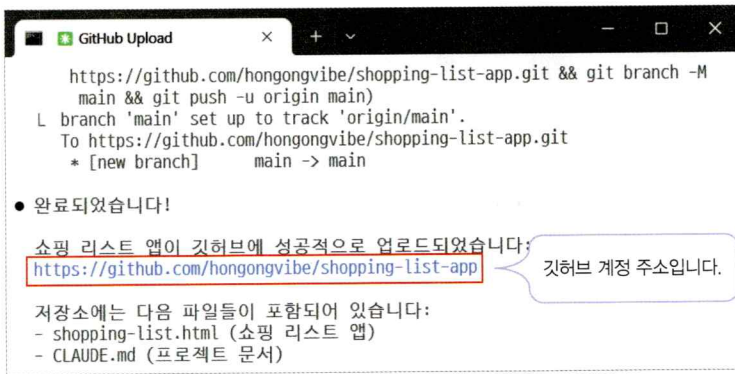
```
git init && git add shopping-list.html CLAUDE.md && git commit -m
"Initial commit: Shopping list web application"
Initialize git repo and commit files
```

Do you want to proceed?

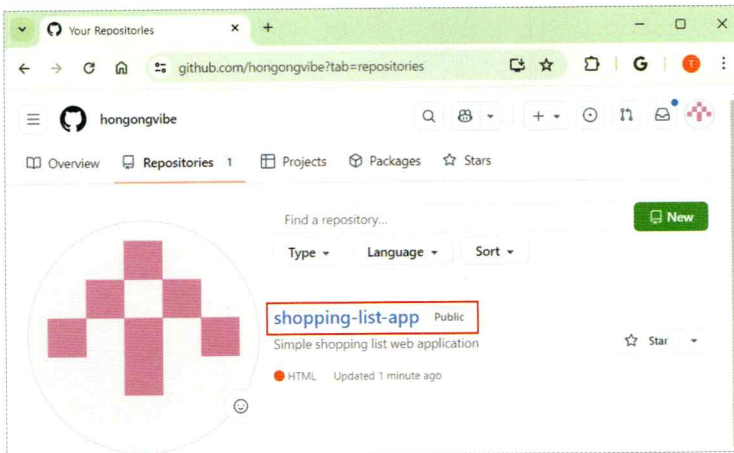
```
> 1. Yes
2. Yes, and don't ask again for similar commands in
  C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-06
3. No, and tell Claude what to do differently (esc)
```

Git 저장소를 초기화하고, 파일을 추가한 뒤 커밋을 생성하는 작업

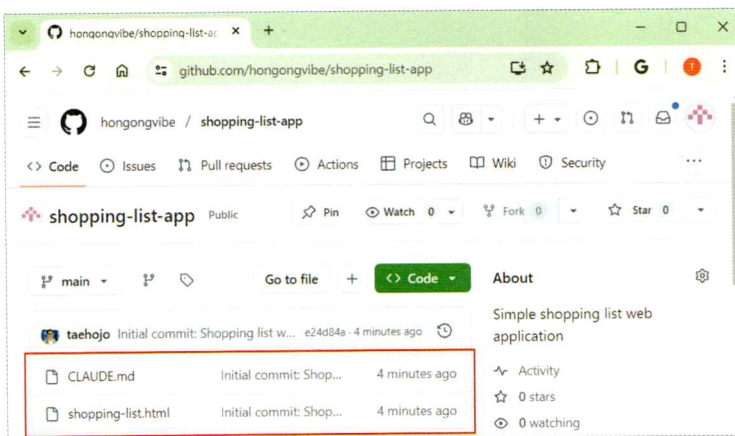
03 쇼핑 리스트 앱이 깃허브에 업로드되었습니다. URL로 이동해 깃허브에서 확인해 보겠습니다.



04 깃허브에 'shopping-list-app' 저장소가 새로 생성되었습니다. 클릭해 보겠습니다.



05 'CLAUDE.md'와 'shopping-list.html' 파일이 성공적으로 업로드되어 있습니다.



지금까지 MCP를 활용해 테스트와 배포까지 전 과정을 AI가 스스로 처리하게 했습니다. Context7 MCP를 통해 항상 최신 상태를 유지하고, Playwright MCP로 기능 테스트를 자동화하며, 깃허브 MCP로 프로젝트의 버전 관리까지 완성했습니다. 이 세 가지 MCP는 AI 개발 환경에서의 자동화 삼총사라고 할 수 있습니다.

이제 개발자는 ‘무엇을 만들지’에 집중하고, 반복적이고 복잡한 기술적 절차는 MCP에게 맡길 수 있게 되었네요. 다음 절에서는 이렇게 만들어진 애플리케이션을 외부 데이터베이스와 연동해 저장하고 관리하는 방법, 그리고 클라우드 환경에서 지속적으로 운영할 수 있는 구조를 살펴보겠습니다.

마무리

▶ 7가지 키워드로 정리하는 핵심 포인트

- **테스트**는 코드가 의도한 대로 실행되는지, 오류가 없는지를 확인하는 과정입니다.
- **버전 관리**는 코드의 변경 이력을 기록하고, 필요할 때 이전 상태로 되돌릴 수 있게 합니다.
- **Context7**은 클로드 코드가 항상 최신 문서를 참고하도록 돕는 지식 연결 도구입니다.
- **Playwright**는 웹 사이트를 자동으로 테스트하고 UI를 검증하는 브라우저 제어 도구입니다.
- 깃허브에 업로드할 파일들을 선택해서 준비 공간에 추가하는 과정을 **추가**^{add}라고 합니다.
- 원격 저장소에 추가된 파일이 확정되면 이를 전송 바로 전 단계인 로컬 저장소에 저장하는데, 이 과정을 **커밋**^{commit}이라고 합니다.
- 로컬 저장소에 커밋된 파일을 원격 저장소로 업로드하는 과정을 **푸시**^{push}라고 합니다.

▶ 확인 문제

1. Playwright MCP가 테스트 완료 후 생성하는 폴더 이름을 고르세요.

- | | |
|-----------------|-------------------|
| ① .test-results | ② .playwright-mcp |
| ③ .automation | ④ .screenshots |

2. 깃허브에 코드를 업로드하는 순서로 올바른 것을 고르세요.

- | | |
|-----------------------|-----------------------|
| ① commit → add → push | ② push → add → commit |
| ③ add → commit → push | ④ add → push → commit |

3. MCP 서버 주소를 찾는 가장 효과적인 방법 두 가지를 설명하세요.

08-3

데이터베이스 연결해 진짜 서비스 만들기

핵심 키워드

Vercel

배포

로컬 스토리지

클라우드 스토리지

Supabase

데이터베이스

테이블

이제 테스트와 버전 관리까지 완료한 쇼핑 리스트 앱을 다른 사람도 사용할 수 있도록 인터넷에 배포해 보겠습니다. 이번 절에서 Vercel을 이용해 웹 애플리케이션을 언제나 켜져 있는 외부 서버에 올려 배포하고, Supabase를 통해 데이터베이스를 연동하는 과정을 실습합니다. 이를 통해 여러분의 앱이 비로소 ‘혼자만의 코드’가 아닌 누구나 접속할 수 있는 진짜 서비스로 완성됩니다.

시작하기 전에

데이터베이스 database, DB는 많은 데이터를 체계적으로 저장하고 관리하는 시스템입니다. 메모장이나 엑셀에 정보를 입력할 수는 있지만, 데이터가 많아질수록 검색이나 수정이 어려워지죠. 데이터베이스는 이런 문제를 해결하기 위해 데이터를 표 형태로 구조화해 저장하고, 필요한 정보를 빠르게 찾아내거나 수정할 수 있게 돕습니다.

웹 애플리케이션에서는 사용자가 입력한 정보를 모두 데이터베이스에 저장합니다. 덕분에 앱을 종료했다 재접속해도 데이터가 유지되고, 여러 기기에서 같은 내용을 불러올 수 있죠. 즉 데이터베이스는 앱의 기억 저장소라 할 수 있습니다.

나만의 웹페이지 배포하기

지금까지 제작한 웹 애플리케이션은 모두 로컬 컴퓨터에서 실행되었고, 모든 데이터도 로컬 환경에 저장되어 있었습니다. 이러한 로컬 환경에서의 테스트는 개발 과정에 있어서 중요한 단계이지만, 개발한 애플리케이션을 다른 사람에게 보여주거나 실제로 쓰게 하려면 **배포** deploy가 필요합니다.

배포는 내 애플리케이션을 외부의 인터넷에 연결된 제3의 서버에 올리는 과정입니다. 사용자들이 앱을 이용하려면 서버가 24시간 켜져 있어야 하죠. 하지만 로컬 환경에서는 내 컴퓨터의 전원이 꺼지면 서버도 함께 종료됩니다. 애플리케이션을 배포하면 항상 켜져 있는 서버에서 앱을 대신 실행해 주기 때문에 언제, 어디서든 웹 주소, 즉 URL로 접속할 수 있죠.

지금부터 08-2절에서 만든 쇼핑 리스트 앱을 많은 사용자가 이용할 수 있도록 인터넷에 배포할 것입니다.

Vercel 계정 만들고 깃허브 연결하기

Vercel은 작성한 코드를 깃허브에 올리면 자동으로 빌드 build, 배포해 주는 클라우드 플랫폼입니다. 간단한 웹 애플리케이션은 깃허브에 업로드하고, Vercel을 사용하면 무료로 배포까지 할 수 있죠. 작업 흐름은 다음과 같습니다.

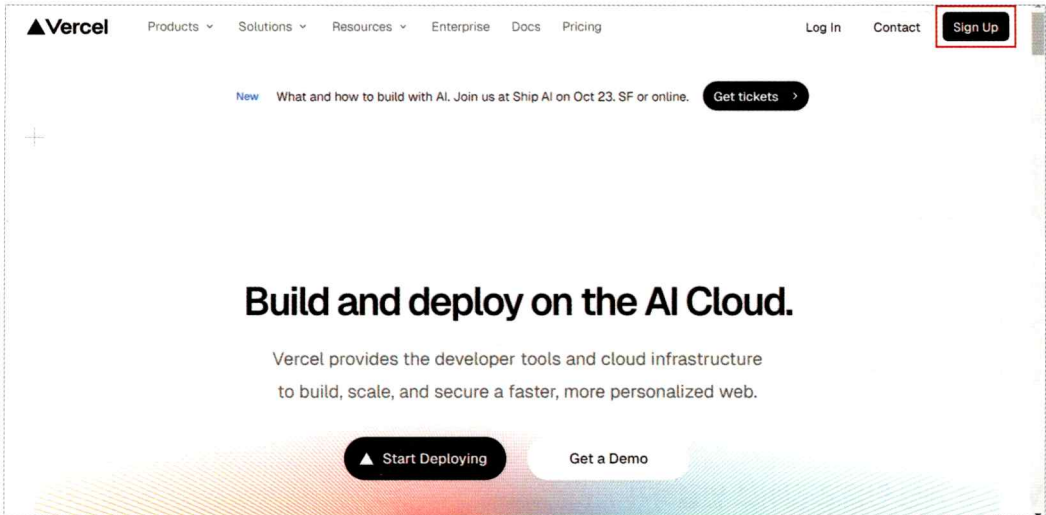
- ❶ 깃허브 저장소 준비
- ❷ 로컬 환경에서 작성한 HTML 파일을 깃허브에 푸시
- ❸ Vercel에 깃허브 인증 허가 후 배포할 저장소 선택
- ❹ 즉시 URL 발급

쇼핑 리스트 앱을 깃허브에서 배포하기 위해 먼저 Vercel 계정을 만들어 보겠습니다.

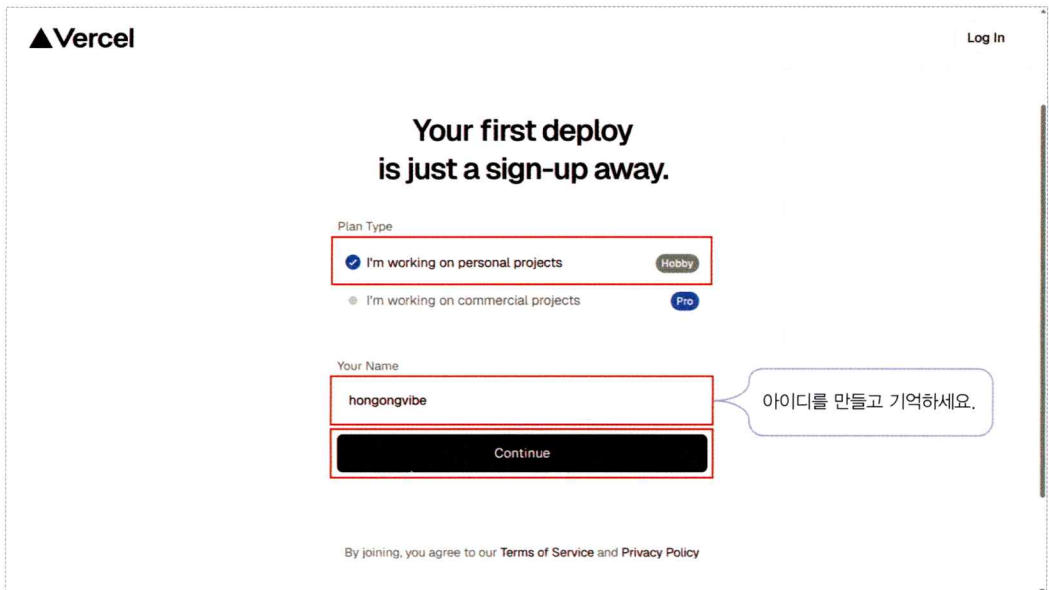


01 Vercel 홈페이지에 접속한 뒤, [Sign Up] 버튼을 클릭해 가입을 시작합니다.

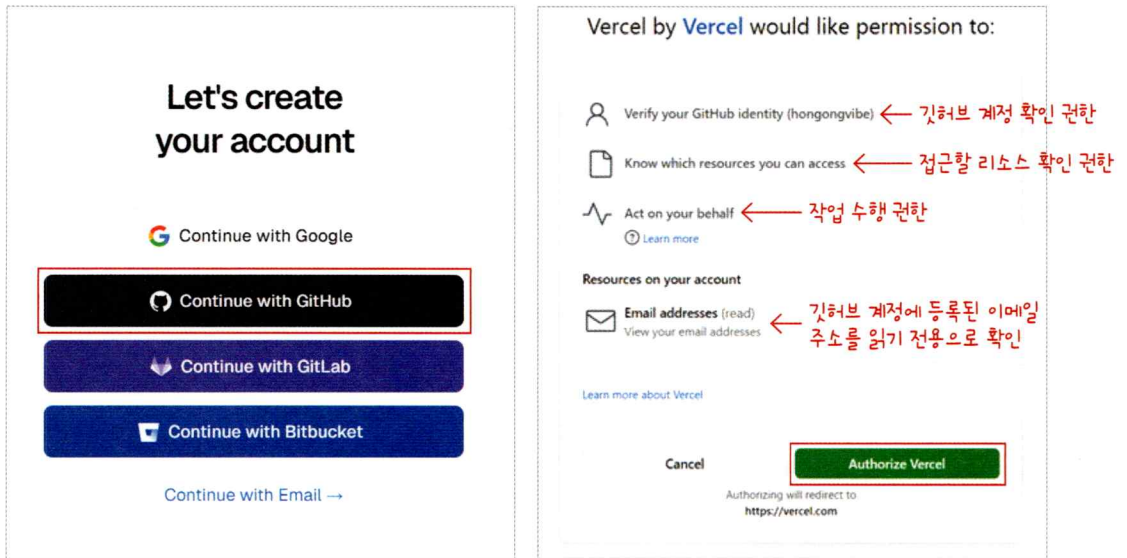
URL <https://www.vercel.com>



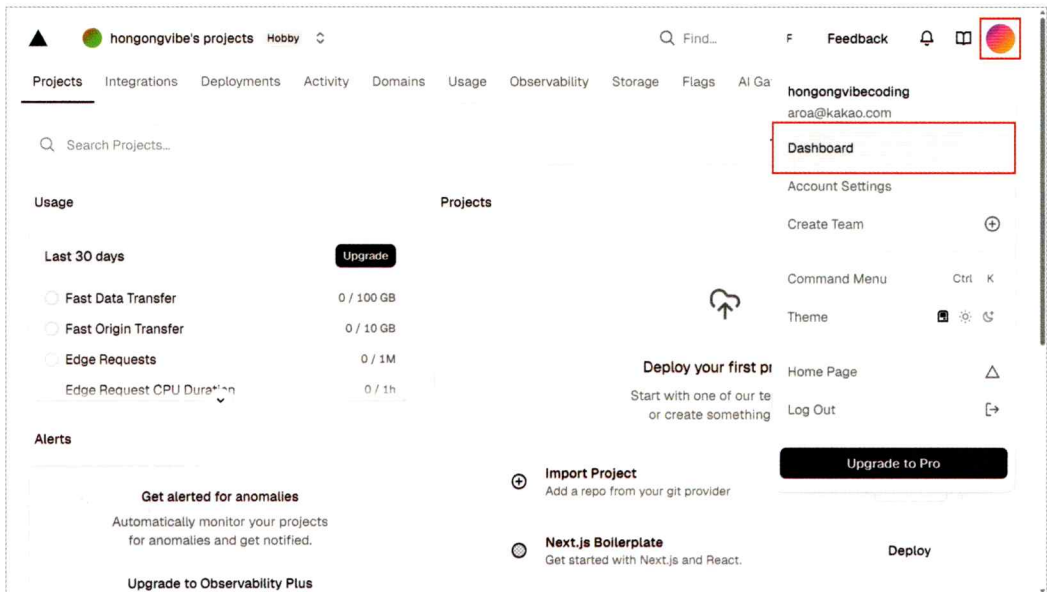
02 'Plan Type(계정 유형)'은 'I'm working on personal projects(개인 프로젝트)'를 선택하고, 'Your Name(이름)' 입력란에 아이디를 입력하세요. 여기서는 'hongongvibe'라고 입력하겠습니다. [Continue] 버튼을 클릭해 진행하세요.



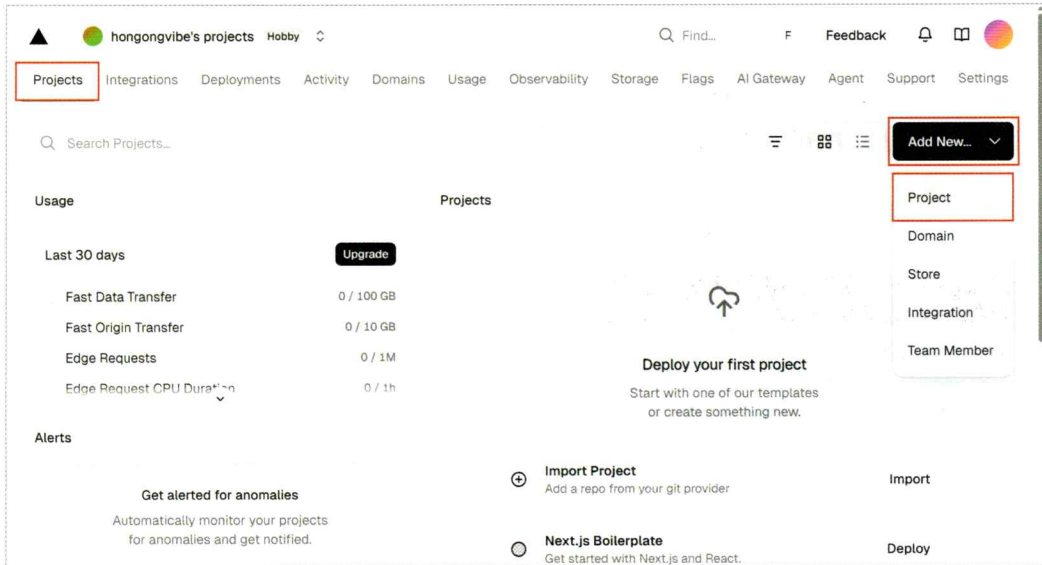
03 Vercel 계정을 생성합니다. 여기서는 [Continue with GitHub]을 선택해 깃허브 계정과 연동했습니다. 이어 열린 창에서 [Authorize Vercel] 버튼을 클릭해 인증하면 계정 생성이 완료됩니다.



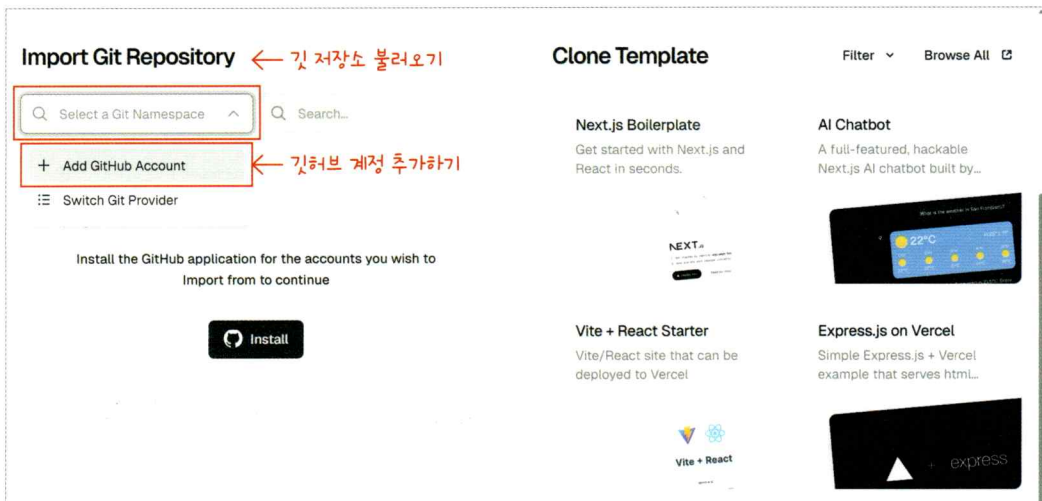
04 Vercel에 로그인하고 우측 상단 프로필을 클릭한 뒤, [Dashboard] 메뉴를 선택합니다.



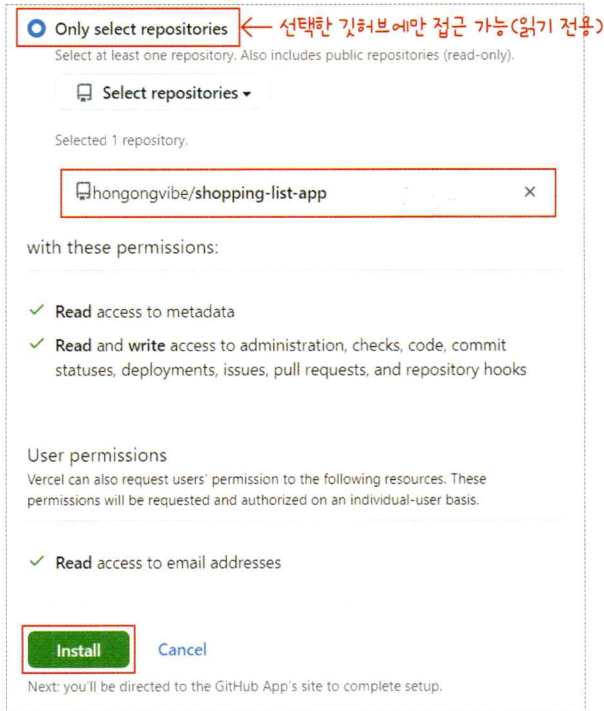
05 [Projects] 탭이 선택된 상태에서 오른쪽 [Add New.. +] 버튼을 클릭한 뒤, 드롭다운 메뉴에서 [Project]를 선택합니다.



06 'Import Git Repository'의 [Select a Git Namespace] 드롭다운 메뉴를 클릭한 뒤, 'Add GitHub Account'를 선택해 내 깃허브와 연동을 시작합니다.



07 선택한 깃허브 저장소에만 접근할 수 있도록 'Only select repositories'에 체크합니다. [Select repositories] 드롭다운 버튼을 눌러 'shopping-list-app'을 선택하고, [Install] 버튼을 눌러 Vercel을 설치합니다.



☒ Only select repositories ← 선택한 깃허브에만 접근 가능(읽기 전용)

Select at least one repository. Also includes public repositories (read-only).

Select repositories ▼

Selected 1 repository.

hongongvibe/shopping-list-app ×

with these permissions:

- ✓ Read access to metadata
- ✓ Read and write access to administration, checks, code, commit statuses, deployments, issues, pull requests, and repository hooks

User permissions

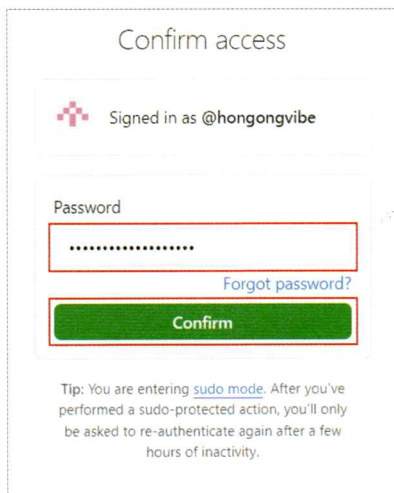
Vercel can also request users' permission to the following resources. These permissions will be requested and authorized on an individual-user basis.

- ✓ Read access to email addresses

Install Cancel

Next: you'll be directed to the GitHub App's site to complete setup.

08 깃허브 비밀번호를 입력해 인증하면, 깃허브 저장소를 불러올 준비가 끝납니다.



Confirm access

Signed in as @hongongvibe

Password

.....

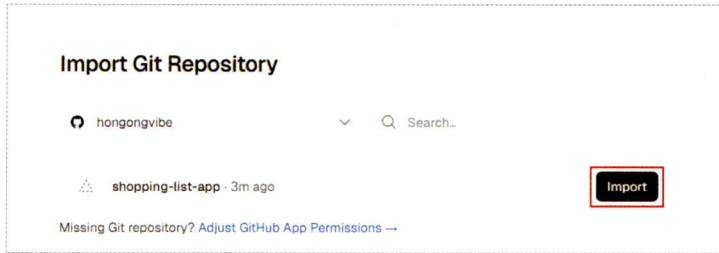
[Forgot password?](#)

Confirm

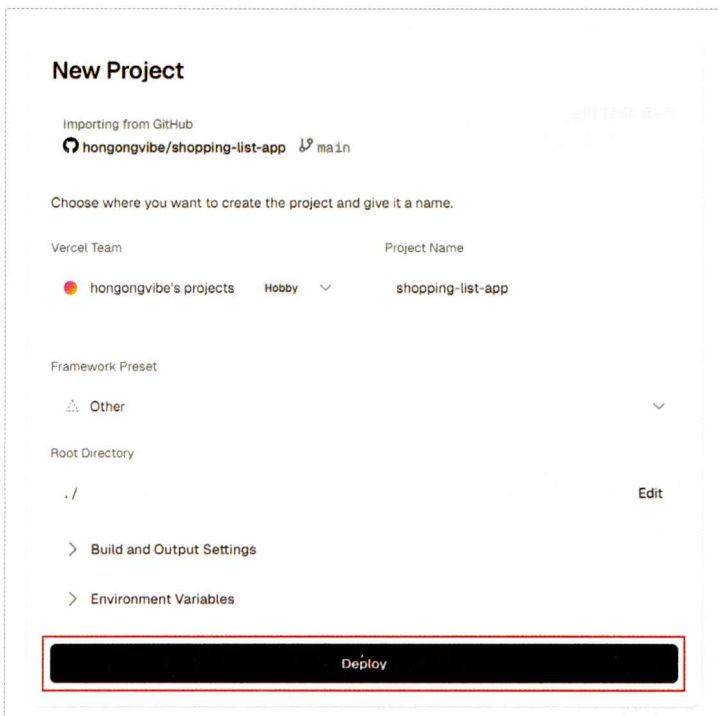
Tip: You are entering [sudo mode](#). After you've performed a sudo-protected action, you'll only be asked to re-authenticate again after a few hours of inactivity.

프로젝트 배포하기

01 Vercel로 돌아와 'New Project' 창의 'Import Git Repository' 하위에 추가된 'shopping-list-app' 저장소를 확인하고, [Import] 버튼을 클릭해 불러옵니다.

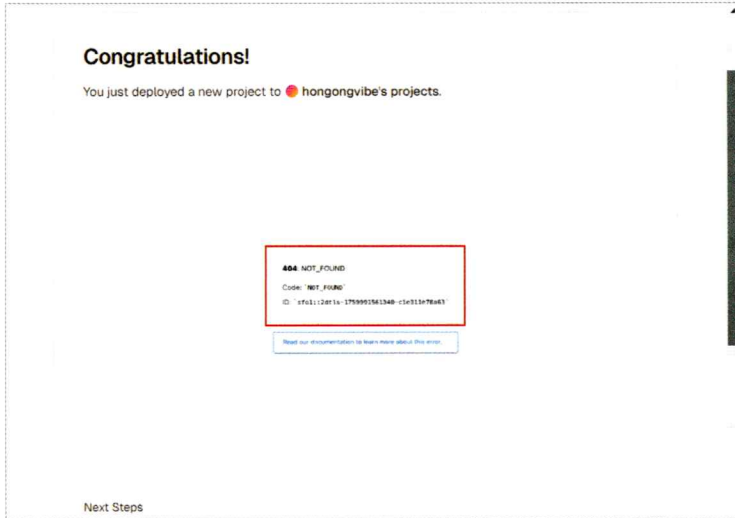


02 새로운 프로젝트를 생성하며 배포할 준비가 되었습니다. [Deploy] 버튼을 클릭하세요.



note 배포가 진행되는 동안 작업 과정을 볼 수 있습니다.

03 잠시 기다리면 프로젝트가 배포됩니다. 만일 미리보기 창에 '404 Not Found'라는 오류 메시지가 보인다면, 이는 깃허브 저장소에 아직 웹사이트의 기본 시작 페이지인 'index.html' 파일이 없기 때문입니다. 쇼핑 리스트 앱이 'index.html'이 아닌 'Shopping-list.html'이라는 이름으로 만들어졌기 때문이죠.



04 클로드 코드를 이용해 'shopping-list.html' 파일의 이름을 'index.html'로 수정하겠습니다. 클로드 코드에 다음과 같이 프롬프트를 입력해 주세요.

프롬프트

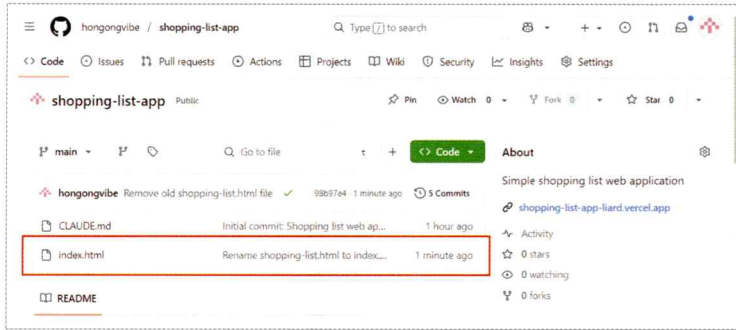
쇼핑 리스트 앱의 shopping-list.html 파일을 index.html로 수정해서 깃허브에 업로드해 줘.

> 쇼핑 리스트 앱의 shopping-list.html 파일을 index.html로 수정해서 깃허브에 업로드 해줘.■

05 완료 메시지를 확인합니다.

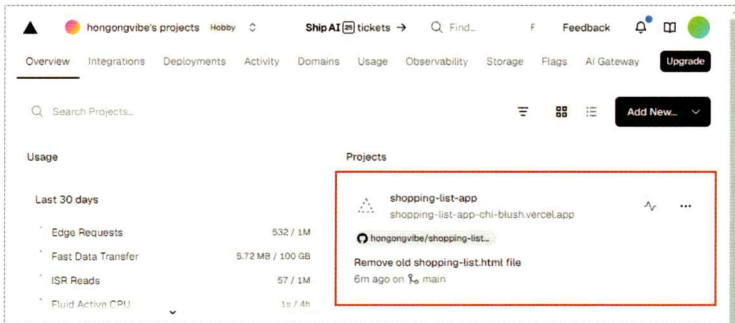


06 깃허브로 돌아가 'shopping-list.html' 파일이 'index.html'로 수정된 것을 확인합니다.



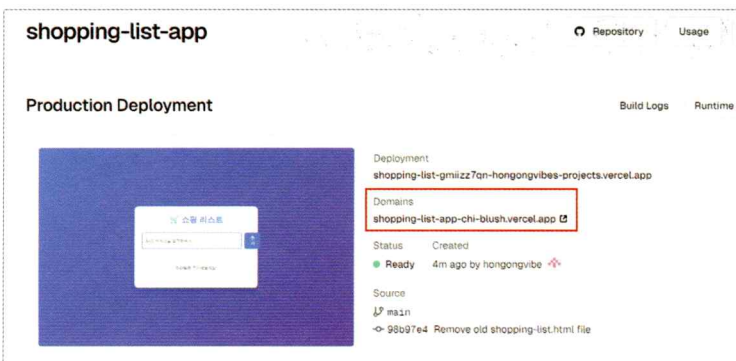
배포된 내 웹사이트 확인하기

01 Vercel의 대시보드 dashboard에서 조금 전 생성된 'shopping-list-app' 프로젝트를 클릭합니다.

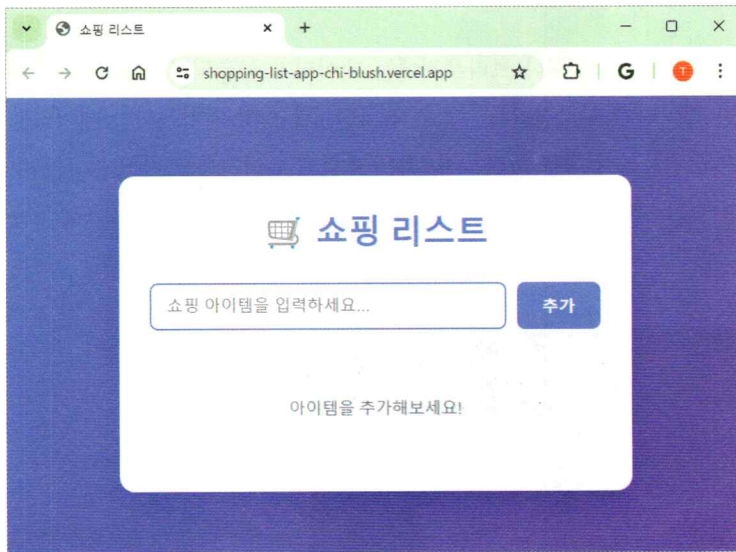


note 대시보드는 Vercel의 우측 상단 프로필을 클릭한 뒤, [Dashboard] 메뉴를 선택해 볼 수 있습니다.

02 배포가 완료된 쇼핑 리스트 앱을 미리보기 창에서 볼 수 있습니다. 미리보기 창 오른쪽의 'Domains' 항목에 배포된 웹 페이지의 URL이 있습니다. 전송하기(📄) 아이콘을 클릭해서 해당 주소에 방문해 봅시다.



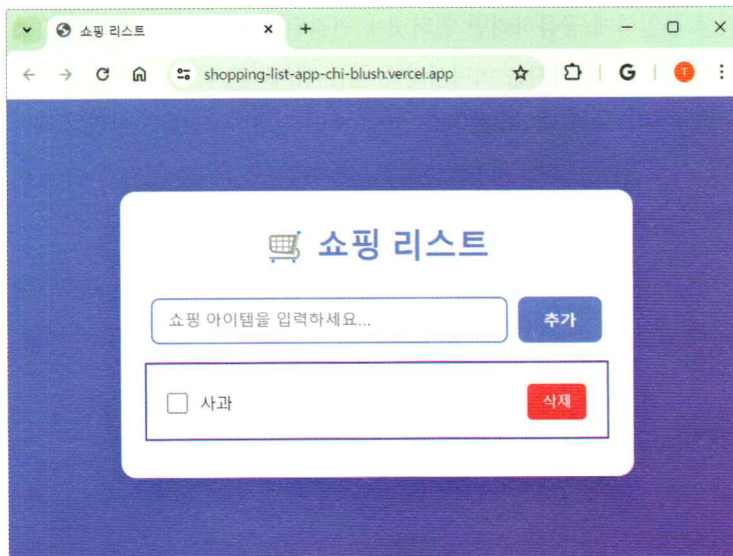
03 쇼핑 리스트 앱이 인터넷에 배포되었습니다. 이제 전 세계의 많은 사용자들이 내가 만든 웹 페이지를 방문할 수 있습니다.



note 이 책에서 배포한 완성된 쇼핑 리스트 앱의 주소는 'https://shopping-list-app-chi-blush.vercel.app'입니다.

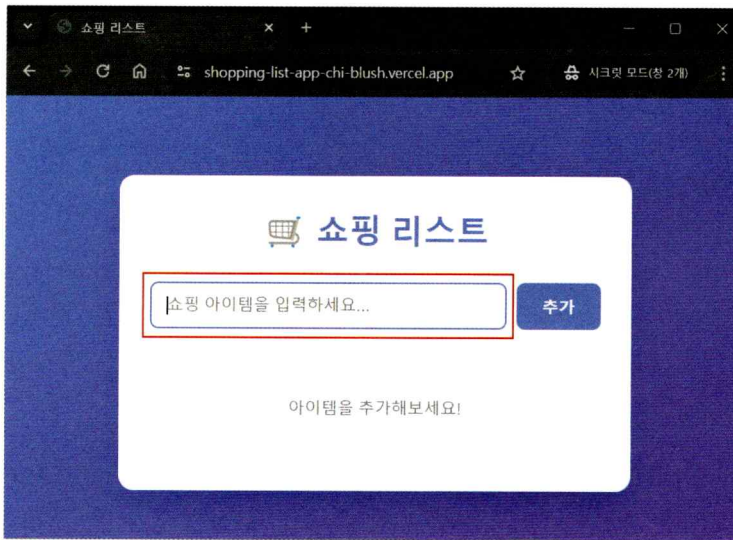
애플리케이션에 데이터베이스 연동하기

이제 배포된 웹 애플리케이션을 테스트해 보겠습니다. 쇼핑 리스트에 입력한 사과 아이템이 잘 보입니다.



그런데 사실 이 웹 애플리케이션은 아직 완성된 것이 아닙니다. 지금 쇼핑 리스트 앱은 데이터를 브라우저의 **로컬 스토리지** localStorage에 저장하기 때문에, 지금 사용하고 있는 이 브라우저에서만 저장된 데이터를 볼 수 있습니다.

로컬 스토리지에는 한계가 있는데, 다른 컴퓨터나 브라우저에는 데이터가 공유되지 않고 브라우저 캐시를 지우거나 기기를 바꾸면 데이터가 사라집니다. 즉 오직 나만, 지금 이 기기에서만 데이터를 볼 수 있는 셈입니다. 실제로 다른 컴퓨터나 다른 브라우저, 혹은 로컬 스토리지를 쓰지 않는 시크릿 모드 secret mode에서 애플리케이션에 접속하면 빈 쇼핑 리스트만 보일 뿐, 저장된 데이터는 보이지 않습니다.



브라우저의 시크릿 모드에서는 쇼핑 아이템 리스트가 텅 비어 있습니다.



다른 기기에서도 저장한 데이터를 동일하게 공유하려면 인터넷에 연결된 외부 저장소, **클라우드 스토리지** cloud storage가 필요합니다. 클라우드 스토리지는 어디서나 동일한 데이터를 불러올 수 있는 중앙 서버 역할을 합니다. 예를 들어, 쇼핑 리스트를 **데이터베이스** database, DB에 저장하고, 앱이 실행될 때마다 클라우드로서 데이터를 읽어오는 것입니다. 이렇게 하면 데스크톱, 노트북, 스마트폰에서 같은 리스트를 확인할 수 있고, 여러 사람이 공유하거나 협업할 수도 있겠죠.

일반적으로 웹 애플리케이션이 데이터를 저장하려면 Oracle, MySQL 같은 데이터베이스 프로그램을 설치하고 직접 서버를 운영해야 합니다. 하지만 이런 전통적인 방식은 설정이 복잡하고, 서버 유지와 보수에 많은 시간이 듭니다.

Supabase는 이런 과정을 간단히 자동화해 주는 클라우드 데이터베이스 플랫폼입니다. 회원가입만 하면 별도의 설치 없이도 데이터베이스를 만들 수 있고, 웹이나 모바일 앱에서 API 형태로 데이터를 주고받을 수 있습니다.

Supabase는 '수퍼베이스'라고 읽습니다.

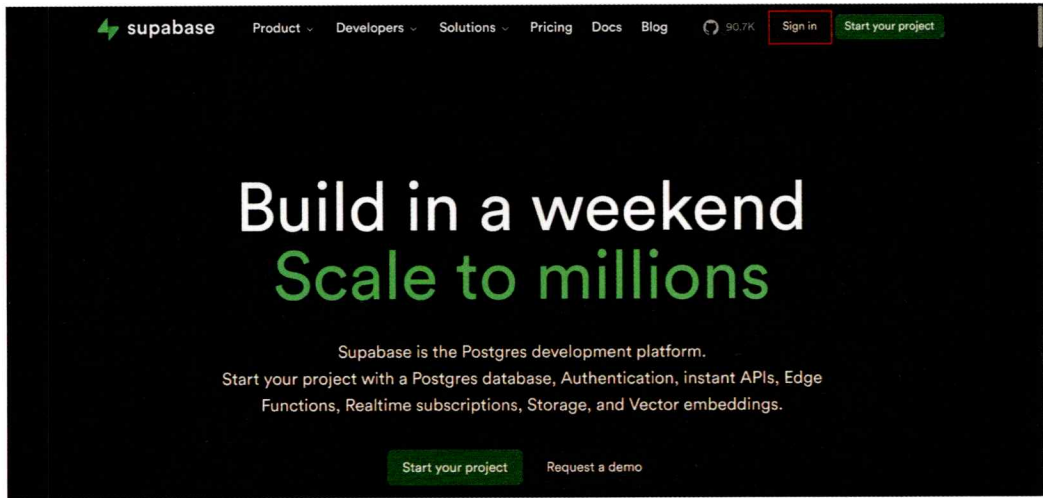
Supabase는 개인 프로젝트나 학습용으로 사용하기 충분한 무료 플랜을 제공하고, 데이터를 저장하면 API가 자동으로 생성되어 직접 서버 코드를 작성할 필요가 없습니다. 또한 데이터를 온라인 서버에 저장하여 여러 사용자가 실시간으로 데이터를 추가하거나 수정해도 자동으로 반영됩니다.

지금부터 Supabase MCP 서버를 사용해서 쇼핑 리스트 웹 앱을 데이터베이스에 연동하고 클라우드 기반 웹 앱으로 확장해 봅시다.

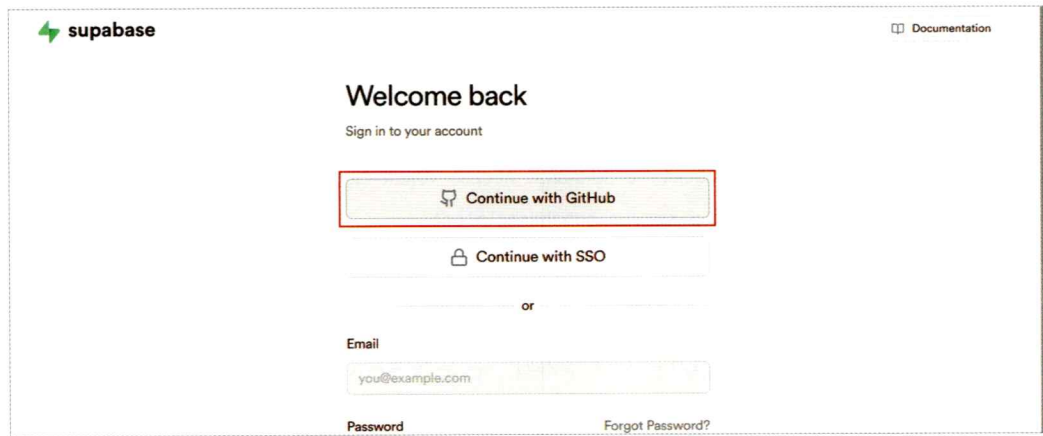
Supabase 가입하고 조직 생성하기

01 Supabase 홈페이지에 접속하세요. 우측 상단의 [Sign in] 버튼을 클릭해 회원가입해 봅시다.

URL www.supabase.com



02 [Continue with GitHub] 버튼을 클릭해 깃허브 계정으로 회원가입합니다.



03 'New organization' 창에서 조직 organization 을 생성할 차례입니다. Supabase에서 말하는 조직은 여러 프로젝트를 묶어 관리하는 폴더와 같은 개념입니다. 각 항목은 다음과 같이 입력한 뒤, [Create organization] 버튼을 클릭하여 조직을 생성합니다.

- **Name**: HonGongVibeCoding's Org(자동 생성됨)
- **Type**: Personal
- **Plan**: Free - \$0/month

note 무료 플랜으로는 프로젝트를 2개까지 만들 수 있습니다. 비활성 상태가 7일간 지속되면 자동으로 일시정지되며, 대시보드의 [Resume] 버튼을 누르면 재시작할 수 있습니다.

Supabase MCP 서버 설치와 API 토큰 발급

이제 Supabase MCP 서버를 설치해 보겠습니다. Supabase는 로컬 MCP 서버와 원격 MCP 서버를 모두 지원합니다. 이 책에서는 로컬 MCP 서버를 설치하겠습니다. 터미널을 실행하고 다음과 같이 설치 명령어를 입력한 뒤, 인증 과정을 거칩니다.

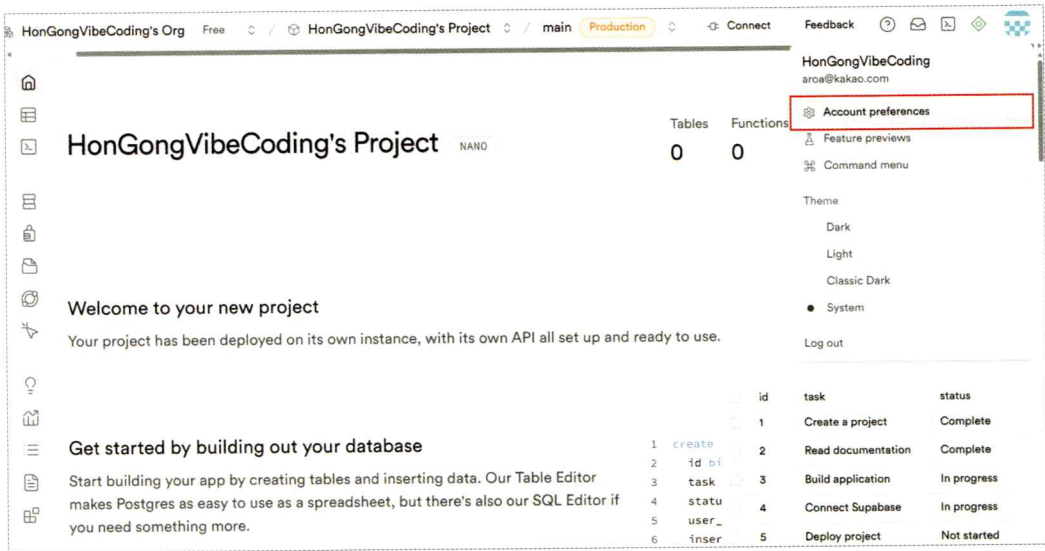
```
claude mcp add supabase -s local -e SUPABASE_ACCESS_TOKEN=<Supabase API 토큰>
-- cmd /c npx -y @supabase/mcp-server-supabase@latest
```

이 명령어는 Supabase사(@supabase)가 만든 MCP 서버(mcp-server-supabase)를 최신 버전(@latest)으로 실행하라는 의미입니다. 새로 추가된 '-e'는 '--env'의 축약형으로, MCP 서버에 전

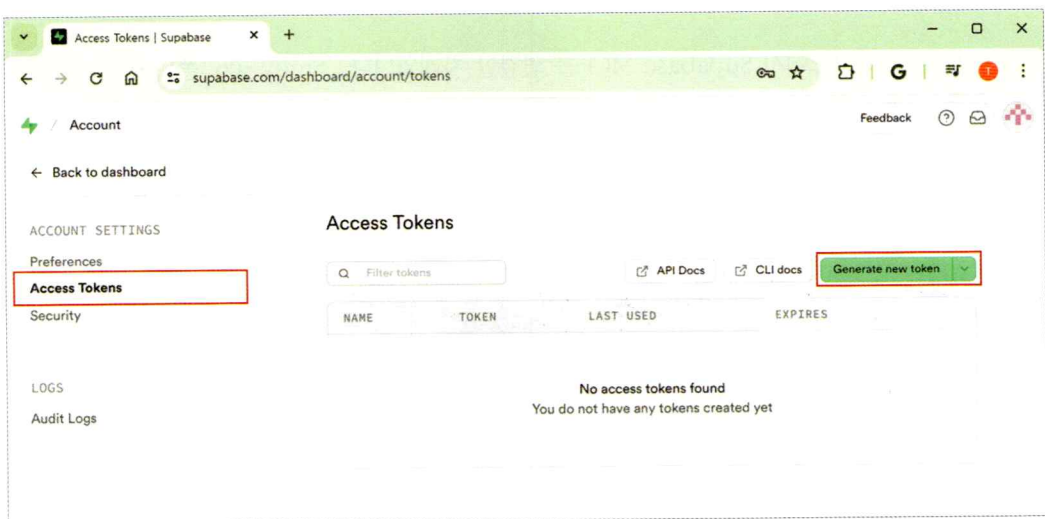
달하는 환경 변수를 설정하는 옵션입니다. 여기서는 Supabase를 인증할 API 토큰이 필요한 상황이므로 토큰을 전달하는 옵션이 추가된 것입니다.

Supabase MCP를 설치하기 위해서는 API 토큰이 필요하다는 사실을 알게 되었네요. 이제 Supabase 홈페이지로 돌아가 API 토큰을 발급받아 봅시다.

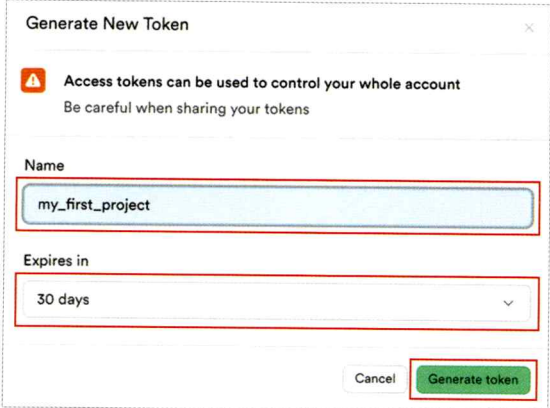
01 Supabase 시작 페이지에서 오른쪽 상단의 프로필을 클릭한 뒤, [Account preferences] 메뉴를 선택합니다.



02 왼쪽 사이드바에서 [Access Tokens]를 클릭하면 토큰 관리 화면으로 이어집니다. 오른쪽의 [Generate new token] 버튼을 클릭하여 새로운 API 토큰을 생성합니다.

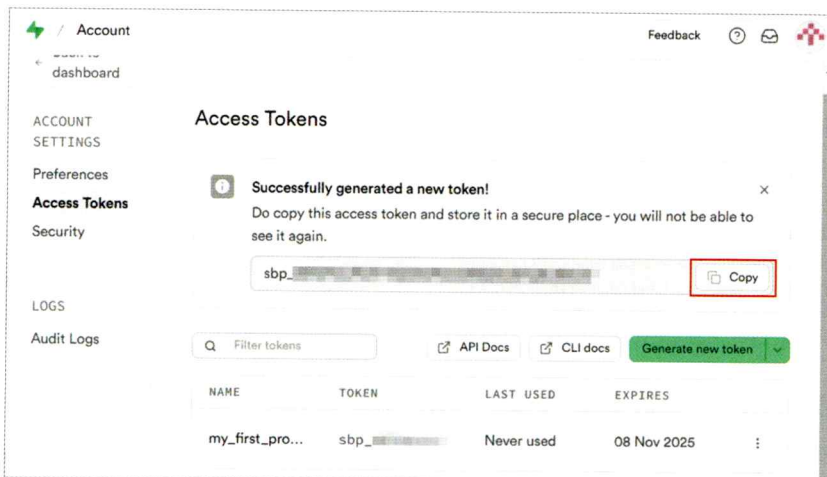


03 'Generate New Token' 창의 'Name' 입력란에는 'my_first_project'를 'Expires in(토큰 유효 기간)'은 '30 days'로 설정합니다. 그리고 나서 [Generate token] 버튼을 클릭해 설정을 완료합니다.



The 'Generate New Token' dialog box contains a warning icon and text: 'Access tokens can be used to control your whole account. Be careful when sharing your tokens.' Below this, there are two input fields: 'Name' with the value 'my_first_project' and 'Expires in' with a dropdown menu set to '30 days'. At the bottom right, there are two buttons: 'Cancel' and 'Generate token'.

04 토큰이 성공적으로 생성되었습니다. [Copy] 버튼을 클릭하여 복사한 후, 안전한 곳에 저장해 둡니다.

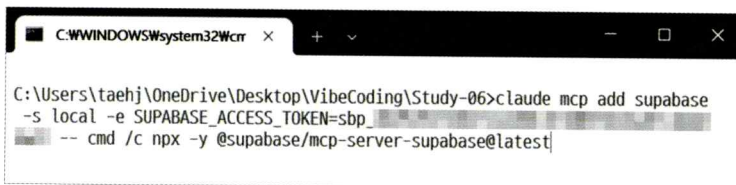


The 'Access Tokens' page shows a success message: 'Successfully generated a new token! Do copy this access token and store it in a secure place - you will not be able to see it again.' Below the message is a text field containing a masked token 'sbp_...' and a 'Copy' button. At the bottom, there is a table with columns: NAME, TOKEN, LAST USED, EXPIRES, and a menu icon.

NAME	TOKEN	LAST USED	EXPIRES
my_first_pro...	sbp_...	Never used	08 Nov 2025

05 이제 클로드 코드에서 Supabase MCP를 연결할 차례입니다. 'Study-06' 폴더에서 터미널을 실행한 후, 다음과 같이 Supabase MCP를 설치합니다.

```
▶ claude mcp add supabase -s local -e SUPABASE_ACCESS_TOKEN=<내 API 토큰> -- cmd /c npx -y @supabase/mcp-server-supabase@latest
```



A terminal window showing the command execution: 'C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-06>claude mcp add supabase -s local -e SUPABASE_ACCESS_TOKEN=sbp_... -- cmd /c npx -y @supabase/mcp-server-supabase@latest'.

note <내 API 토큰> 자리에 발급받은 Supabase API 토큰을 입력하세요.

06 MCP 서버 설치가 완료되면 'claude mcp list' 명령어를 입력해 MCP가 잘 설치되었는지 확인해 봅니다. 'supabase'가 목록에 잘 추가되었네요.

```
C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-06>claude mcp list
Checking MCP server health...

notion: https://mcp.notion.com/mcp (HTTP) - ✓ Connected
sequential-thinking: cmd /c npx -y @modelcontextprotocol/server-sequential-
thinking@latest - ✓ Connected
playwright: cmd /c npx @playwright/mcp@latest - ✓ Connected
github: https://api.githubcopilot.com/mcp (HTTP) - ✓ Connected
context7: https://mcp.context7.com/mcp (HTTP) - ✓ Connected
vercel: https://mcp.vercel.com (HTTP) - ✓ Connected
supabase: cmd /c npx -y @supabase/mcp-server-supabase@latest - ✓ Connected

C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-06>
```

쇼핑 리스트 앱 진짜 서비스로 업그레이드하기

이제 쇼핑 리스트 앱을 Supabase MCP를 이용해서 데이터베이스와 연동할 차례입니다.

클로드 코드로 데이터베이스 자동 연동하기

01 클로드 코드를 실행하고 다음과 같이 입력합니다.

프롬프트

Supabase mcp를 이용해서 우리의 쇼핑 리스트 앱을 데이터베이스와 연동해 줘. shopping_items라는 테이블을 만들고, 현재 로컬 스토리지에 저장되던 데이터를 Supabase 데이터베이스에 저장하도록 코드를 수정해 줘. 수정이 완료되면 깃허브에 commit하고 push해 줘.

```
C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-06>claude

Claude Code v2.0.11
Sonnet 4.5 · Claude API
C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-06
```

```
> Supabase mcp를 이용해서 우리의 쇼핑 리스트 앱을 데이터베이스와 연동해
줘. shopping_items라는 테이블을 만들고, 현재 로컬 스토리지에 저장되던
데이터를 Supabase 데이터베이스에 저장하도록 코드를 수정해 줘. 수정이
완료되면 깃허브에 commit하고 push해 줘. █
```

데이터베이스의 **테이블** table은 쉽게 말하면 데이터를 표 형태로 저장한 구조입니다. 데이터베이스가 여러 엑셀 파일을 관리하는 프로그램이라면, 테이블은 데이터베이스 안에 있는 개별 엑셀 시트와 같죠.

예를 들어 shopping_items 테이블의 행row에는 쇼핑 아이템이, 열column은 아이템의 속성이 저장됩니다. 각 열에는 고유 번호(id), 상품명(name), 구매 완료 여부(completed), 생성 시간(created_at)이 저장됩니다.

id	name	completed	created_at
1	사과	False	2025-10-29 15:30:00
2	우유	True	2025-10-29 15:32:10
3	빵	false	2025-10-29 15:35:22

이렇게 구조화된 형태로 데이터를 저장하면 필요한 정보를 빠르게 찾고 수정할 수 있습니다.

02 Supabase MCP가 작동됩니다. '1. Yes'를 눌러 계속 진행시키겠습니다.

- Supabase MCP를 사용하여 쇼핑 리스트 앱을 데이터베이스와 연동하겠습니다. 먼저 작업을 계획하고 시작하겠습니다.

Tool use

```
supabase - List organizations() (MCP)
Lists all organizations that the user is a member of.
```

Do you want to proceed?

> 1. Yes

2. Yes, and don't ask again for supabase - List organizations commands
in C:\Users\taehj\OneDrive\Desktop\VibeCoding\Study-06
3. No, and tell Claude what to do differently (esc)

03 중간에 Supabase에 프로젝트를 새로 만든다는 메시지와 함께 무료 플랜임을 확인하는 메시지가 나옵니다. 내용을 확인하고 계속 진행하겠습니다.

- 새로운 Supabase 프로젝트를 생성하려면 비용이 월 \$0입니다 (무료 플랜). 프로젝트를 생성해도 괜찮으신가요?

프로젝트 이름은 "shopping-list-app"으로 하고, 가장 가까운 리전인 ap-northeast-2 (서울)에 생성하겠습니다.

> 진행시켜

04 Supabase MCP를 연동할 수 있도록 'index.html' 파일을 자동으로 수정합니다.

```
Supabase Integration x + - □ x
● 완벽합니다! 이제 코드를 Supabase를 사용하도록 업데이트하겠습니다.
● Update(index.html)
  L Updated index.html with 72 additions and 30 removals
    174     </ul>
    175   </div>
    176
    177 + <script src="https://cdn.jsdelivr.net/npm/@supabas
    + e/supabase-js@2"></script>
    178   <script>
    179 + // Supabase 초기화
    180 + const SUPABASE_URL =
    + 'https://klxwcnrhasnoxadsqgbx.supabase.co';
    181 + const SUPABASE_ANON_KEY =
    + 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYy
    +
    + .evn5Uumd641NNJXYUorYVro8Pw2IUC0eBsxgmK1Mtuy';
    182 + const supabase =
    + window.supabase.createClient(SUPABASE_URL,
    + SUPABASE_ANON_KEY);
    183 +
```

05 shopping_items 테이블을 생성하고 id, name 등 필요한 필드를 포함해 성공적으로 데이터베이스를 연동했습니다.

```
Supabase Integration x + - □ x
● 완료되었습니다! 쇼핑 리스트 앱이 성공적으로 Supabase 데이터베이스와 연동되었습니다.

완료된 작업:

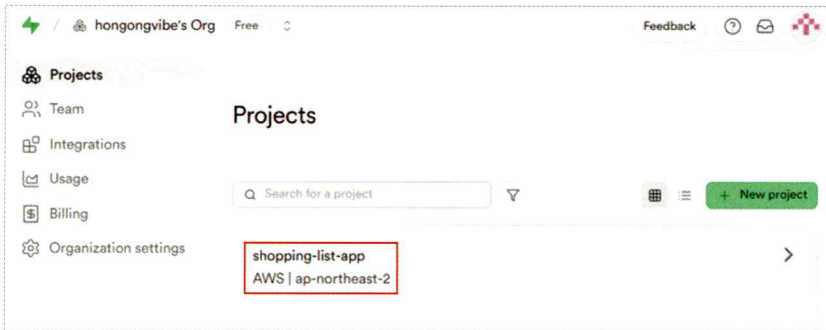
1. ✓ Supabase 프로젝트 생성: "shopping-list-app" 프로젝트를
ap-northeast-2 (서울) 리전에 생성
2. ✓ 데이터베이스 테이블 생성: shopping_items 테이블을 생성하고 다음
필드 포함:
  - id (UUID, 기본키)
  - name (TEXT, 아이템 이름)
  - completed (BOOLEAN, 완료 여부)
  - created_at, updated_at (타임스탬프)
3. ✓ 코드 업데이트:
  - localStorage를 Supabase로 완전히 교체
  - Supabase JavaScript SDK를 CDN으로 추가
  - 모든 CRUD 작업(추가, 읽기, 업데이트, 삭제)을 async/await로 구현
  - 에러 핸들링 추가
4. ✓ GitHub 커밋 & 푸시: 변경사항을 커밋하고 원격 저장소에 푸시 완료

이제 쇼핑 리스트 데이터가 클라우드 데이터베이스에 저장되어 어떤
디바이스에서든 접근할 수 있습니다!
```

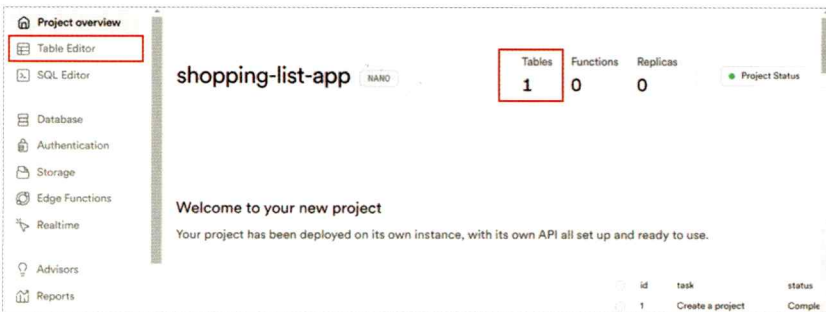
데이터베이스 테이블 생성 확인하기

이제 Supabase 홈페이지에서 데이터베이스가 어떻게 생성되었는지 확인해 보겠습니다.

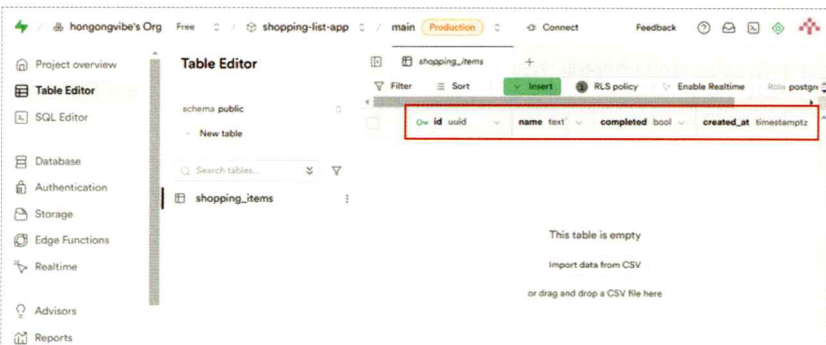
01 대시보드에 'shopping-list-app'이라는 이름의 새 프로젝트가 생성되었습니다. 클릭해서 상세 정보를 확인해 보겠습니다.



02 우측 상단에 'Tables'에 숫자 '1'이 표시된 것으로 새로운 하나의 테이블이 추가되었음을 알 수 있습니다. 왼쪽 사이드바에서 [Table Editor] 메뉴를 클릭하세요.



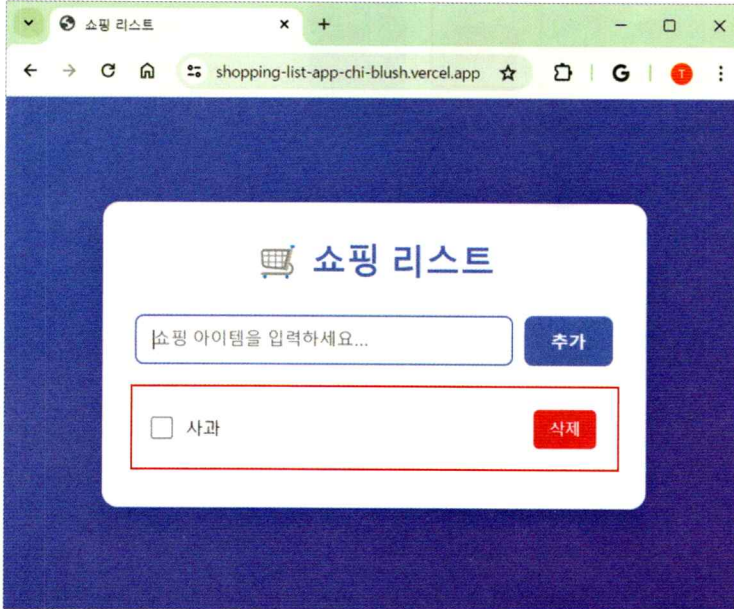
03 [Table Editor] 창에서 [shopping_items] 테이블을 클릭하면 id, name, completed, created_at 열(column)이 생성된 것을 확인할 수 있습니다.



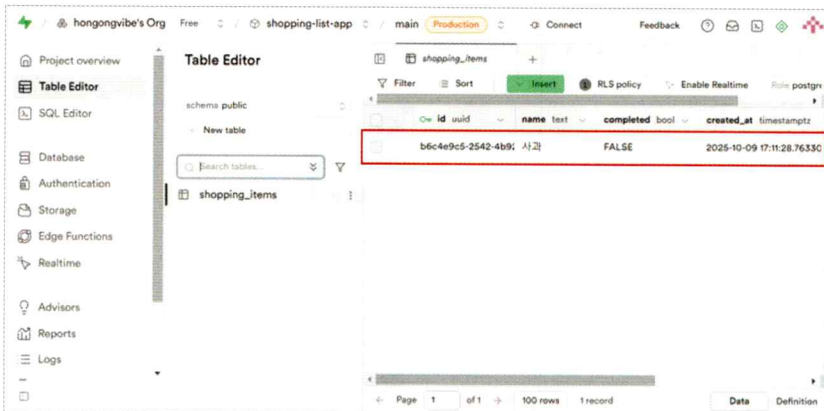
데이터베이스와의 실시간 동기화 확인하기

Supabase 데이터베이스에 테이블이 잘 생성되어 있는 것을 확인했으니, 이제 실제 앱에서 작동하는지 테스트해 봅시다.

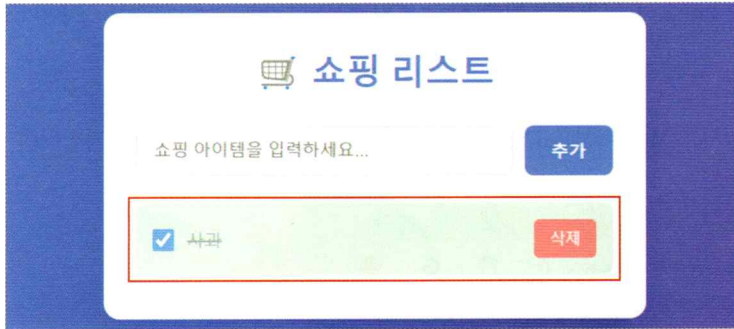
01 쇼핑 리스트 앱에 접속해, '사과'를 입력하고 [추가] 버튼을 클릭해 리스트에 아이템을 추가합니다.



02 Supabase의 [Table Editor] 창을 새로고침하면 '사과' 행이 추가되었음을 확인할 수 있습니다.



03 쇼핑 리스트 앱에서 '사과' 아이템에 체크 박스를 클릭해 완료 상태로 만듭니다.



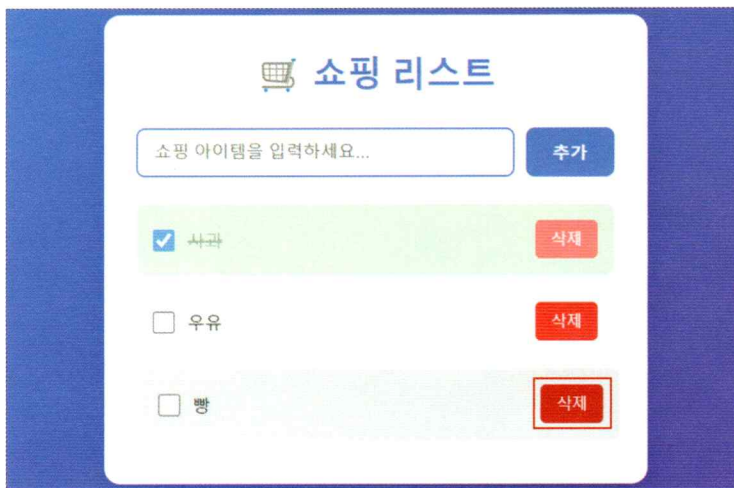
04 테이블의 completed 열이 'TRUE'로 바뀝니다.

	id	uuid	name	text	completed	bool	created_at	timestampz
☐	b6c4e9c5-2542-4b9f		사과		TRUE		2025-10-09 17:11:28.76330	

05 쇼핑 리스트 앱에 다른 아이템들도 추가하고, 데이터베이스의 테이블을 확인해 보세요. 추가한 아이템이 행에 추가됩니다.

	id	uuid	name	text	completed	bool	created_at	timestampz
☐	85f9d216-0c4b-4a7e		빵		FALSE		2025-10-09 17:18:26.2059	
☐	b6c4e9c5-2542-4b9f		사과		TRUE		2025-10-09 17:11:28.76330	
☐	dbd4e1ca-b348-4bef		우유		FALSE		2025-10-09 17:16:30.57163	

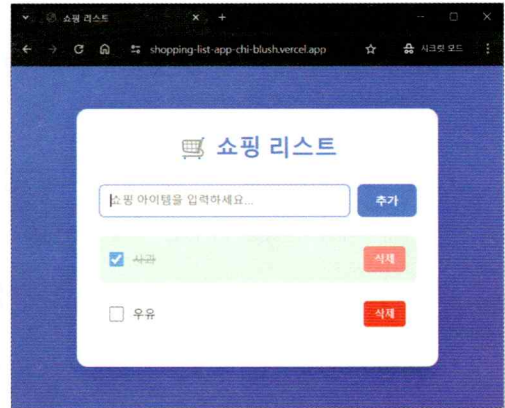
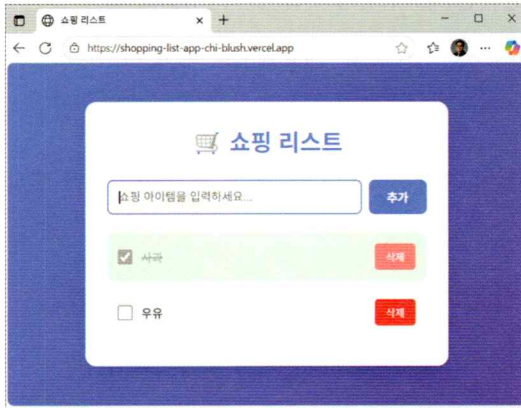
06 이번엔 아이템을 삭제해 보겠습니다. 쇼핑 리스트 앱에서 '빵' 아이템을 삭제합니다.



07 테이블에서 ‘빵’ 행이 삭제되었습니다.

☐	id uuid	name text	completed bool	created_at timestampz
☐	b6c4e9c5-2542-4b94	사과	TRUE	2025-10-09 17:11:28.76330
☐	dbd4e1ca-b348-4bef	우유	FALSE	2025-10-09 17:16:30.57163

08 다른 컴퓨터나 브라우저, 시크릿 모드에서도 역시 잘 작동하는 것을 확인합니다.



이제 우리의 쇼핑 리스트 앱은 단순히 로컬 환경에서만 작동하는 연습용 프로젝트를 넘어, 인터넷을 통해 누구나 접속할 수 있는 실제 웹 서비스로 거듭났습니다. Vercel을 이용한 자동 배포와 Supabase를 통한 데이터베이스 연동으로, 앱은 항상 최신 상태를 유지하며 여러 사용자와 실시간으로 데이터를 공유할 수 있습니다. 이 과정을 통해 우리는 AI가 만든 코드를 실제 서비스로 전환하는 전 과정을 완성했습니다.

▶ 6가지 키워드로 정리하는 핵심 포인트

- **배포**는 내 애플리케이션을 외부의 인터넷에 연결된 제3의 서버에 올리는 과정입니다.
- **Vercel**은 작성한 코드를 깃허브에 올리면 자동으로 빌드^{build}, 배포해주는 클라우드 플랫폼입니다.
- **로컬 스토리지**에 저장된 데이터는 다른 컴퓨터나 브라우저에는 공유되지 않고 브라우저 캐시를 지우거나 기기를 바꾸면 사라집니다.
- **클라우드 스토리지**는 어디서나 동일한 데이터를 불러올 수 있는 중앙 서버 역할을 합니다.
- **Supabase**는 데이터베이스 구축을 자동화해 주는 클라우드 데이터베이스 플랫폼입니다.
- 데이터베이스의 **테이블**은 쉽게 말하면 데이터를 표 형태로 저장한 구조입니다. **데이터베이스**가 여러 엑셀 파일을 관리하는 프로그램이라면, 테이블은 데이터베이스 안에 있는 개별 엑셀 시트와 같습니다.

▶ 확인 문제

1. 로컬 스토리지와 클라우드 데이터베이스의 가장 큰 차이점은 무엇인가요?

- ① 로컬 스토리지가 더 빠르다.
- ② 데이터베이스는 여러 기기에서 데이터를 공유할 수 있다.
- ③ 로컬 스토리지가 더 안전하다.
- ④ 데이터베이스는 무료가 없다.

2. Vercel 배포 시 404 에러가 발생한 이유와 해결 방법으로 옳은 것을 고르세요.

- ① 서버 오류 - 재시작 필요
- ② index.html 파일 부재 - 파일명 변경 필요
- ③ 인증 실패 - 재로그인 필요
- ④ 네트워크 오류 - 연결 확인 필요

3. Supabase의 테이블과 엑셀 시트의 유사점과 차이점을 설명하세요.

혼자 공부하는 바이브 코딩의 여정은 이제 시작
입니다. 여러분이 직접, 시와 함께 새로운 코딩의
세계를 만들어 갈 차례입니다.



01-1 나의 코딩 파트너, AI 어시스턴트

1. 답

구분	설명	대표 도구
웹 기반 챗봇	설치 없이 웹에서 바로 사용할 수 있는 방식, 복사-붙여넣기 필요	클로드, 챗GPT, 제미니
모델 선택형	여러 언어 모델 중에서 선택해 쓸 수 있는 방식, 이중 비용 부담 가능성 있음	커서(Cursor), 리플릿(replit), 윈드서프(Windsurf)
전용 모델형	특정 언어 모델을 설치해 사용하는 방식, 로컬 파일 직접 접근 가능	클로드 코드(Claude code), 제미니 CLI(Gemini CLI)

2. 답 ① 바이트 코딩 ② AI 어시스턴트 ③ 모델 선택형 도구 ④ 전용 모델형 도구

01-2 AI로 나만의 첫 웹페이지 만들기

1. 답 ② 뉴럴 넷 → 딥러닝 → 트랜스포머 → LLM

2. 답 ④ 도구 사용료와 AI 모델 비용을 모두 지불한다.

3. 답 ② 아티팩트(Artifacts)

4. 답 게시

5. 답 Customize

02-1 AI를 깨우는 프롬프트의 비밀

1. 답 ③ 창의성: 결과물이 놀라움을 주도록 요구한다.

2. 답 5W1H

3. 답 ② What(무엇을), How(어떻게)

4. 답 ② “오늘의 날씨와 주요 뉴스를 보여주는 나만의 시작 홈페이지를 만들고 싶어.”

이유: 필요한 기능(날씨, 뉴스)을 명시해 AI가 해야 할 일을 구체적으로 인식할 수 있기 때문이다.

02-2 실전! 마케팅 포트폴리오 완성하기

1. 답 ③ 기본 틀 잡기 → 핵심 기능 넣기 → 디자인 입히기 → 완성도 점검
2. 답 ② 모든 기능을 한 번에 요청하여 시간을 절약한다
3. 답 ④ 자동으로 디자인이 개선된다.

03-1 클로드 코드 설치하기

1. 답 ② Node.js와 Git
2. 답 ③ cmd
3. 답 ③ 시스템 전체에서 사용 가능하도록 설치

03-2 손글씨 인식 프로그램 만들기

1. 답 ③ 클로드 코드가 자동으로 최적 폴더를 선택
2. 답 ③ accuracy는 높을수록, loss는 낮을수록 성능이 좋다.
3. 답 ④ Never allow modifications (수정 절대 불가)

03-3 CLAUDE.md로 프로그램 확장하기

1. 답 ③ 이전 대화 내용 전체 저장
2. 답 ② /init
3. 답 ② 저장 위치를 선택하는 프롬프트를 표시한 후 저장
4. 답 ③ 토큰 사용량이 자동으로 감소

04-1 단계별 프롬프트와 함께 클로드 코드 명령어 익히기

1. 답

설명	명령어
프로젝트 구조를 분석하고 CLAUDE.md 파일을 자동 생성	/init
현재 세션의 토큰 사용량과 남은 컨텍스트 용량 확인	/status
사용할 AI 모델을 선택하거나 Default 옵션으로 변경	/model
사용 가능한 모든 명령어 목록과 설명을 확인	/help

2. 답 ① CLAUDE.md

② PRD (Product Requirement Document)

③ JSON

04-2 작업 재개와 효율성 높이기

1. 답 ④ **Ctrl** + **R** 은 /context 명령어의 단축키이다.🔍 **Ctrl**+**R** 키는 /context 명령어와 별개로, 컨텍스트 요약을 보는 단축키입니다.

2. 답

상황	명령어 / 옵션
여러 대화 중 특정 대화를 선택해서 재개하고 싶을 때	claude --resume 또는 /resume
가장 최근 대화를 즉시 이어서 작업하고 싶을 때	claude --continue 또는 claude -c
현재 컨텍스트 사용량을 시각적으로 확인하고 싶을 때	/context
대화가 길어져서 압축이 필요할 때	/compact
파일 수정 승인을 매번 누르기 번거로울 때	Shift + Tab

04-3 프로젝트 개선과 작업 관리

1. **답** ③ /config는 클로드 코드의 전역 설정만 변경할 수 있다.

2. **답**

상황	명령
web_version 폴더 구조를 확인하고 싶을 때	@web_version
script.js 파일에서 특정 함수를 찾고 싶을 때	@script.js
클로드 코드 설치 상태를 점검하고 싶을 때	/doctor
작업한 대화 내용을 파일로 저장하고 싶을 때	/export 후 파일 저장 옵션 선택
다크 모드로 변경하고 싶을 때	/config에서 Theme 변경
자동 압축 기능을 끄고 싶을 때	/config에서 Auto-compact를 false로 변경

05-1 할루시네이션 없는 AI 콘텐츠 만들기

1. **답** ③ /clear는 대화를 요약하여 압축하고 컨텍스트를 유지한다.

 /clear는 대화를 완전히 초기화하는 명령어입니다. 컨텍스트를 요약하여 유지하는 명령어는 /compact입니다.

2. **답**

상황	대응
퀴즈 게임의 검증 가이드라인을 저장하고 싶을 때	/memory 선택 후 Project memory 선택
이전 대화가 새 작업에 영향을 줄 것 같을 때	/clear 실행
대화가 길어져서 토큰을 절약하고 싶을 때	/compact 실행
"세계에서 가장 긴 강" 같은 애매한 문제를 발견했을 때	측정 기준이나 시점을 명시하도록 가이드라인 추가

05-2 자동화로 개발 효율 높이기

1. **답** ③ 명령어 생성 후 즉시 사용 가능하다.

 새로 생성한 명령어를 사용하려면 클로드 코드를 재시작해야 합니다.

2. **답** • 프로젝트 전용 커스텀 명령어가 저장되어야 할 위치는 **.claude/commands** 폴더이다.

• 새 명령어 작성 후 사용가능하게 하려면 클로드 코드를 **재시작**해야 한다.

- 커스텀 명령어에서 특정 번호 범위의 문제만 검토하려면 \$1, \$2와 같이 플레이스홀더를 사용한다.
- 커스텀 명령어에서 사용자 입력값을 받으려면 \$ARGUMENTS와 같은 매개 변수를 사용한다.
- ~/.claude/commands에 만드는 명령어는 모든 프로젝트에서 사용 가능하다.
- .claude/commands/에 만드는 명령어는 현재 프로젝트에서 사용 가능하다.

05-3 명령어 체이닝으로 익히는 유지보수 전략

1. 답 ② 첫 번째 명령이 실패해도 두 번째 명령은 실행된다.
 🔍 첫 번째 명령이 성공해야 두 번째 명령이 실행됩니다.
2. 답 ① 두 명령어를 연결하여 순차 실행하려면 && 연산자를 사용한다.
 • 새 기능을 추가할 때는 새로운 커스텀 명령어를 만들고 통합 명령어에 추가한다.
 • 커스텀 명령어는 md(마크다운) 파일 형식으로 저장한다.

06-1 클로드 코드에서 API 설정하기

1. 답 ① API 키 ② 모델명
2. 답 ③ 코드에 직접 API 키 하드코딩
3. 답 ③ 호출한 코드 내용

06-2 클로드 코드와 API로 만드는 ‘냉장고를 부탁해’

1. 답 ① .env 파일의 OPENROUTER_API_KEY 존재 여부
 ② DeepSeek 모델명의 정확성
2. 답 ③ Streamlit은 반드시 프론트엔드와 백엔드를 분리해야 한다.
 🔍 Streamlit은 하나의 파이썬 코드 안에서 UI와 로직을 함께 작성하기 위해 사용하는 웹 프레임워크입니다.
3. 답 API 키를 .env 파일에 저장하고 해당 파일을 .gitignore 파일에 추가하여 깃허브 업로드에서 제외합니다.

07-1 클로드 코드의 AI 에이전트 이해하기

1. **답** ② 에이전트는 독립적인 AI 모델을 선택할 수 있지만 커스텀 명령어는 메인 세션의 모델을 공유한다.
2. **답** ② 메인 세션에서 사용 중인 모델을 그대로 상속받는다.
3. **답** ③ 발견한 버그 수정
④ 새로운 테스트 파일 생성

 Read-only는 읽기 전용 권한으로 코드, 설정, 파일의 내용을 읽고 디렉터리 구조를 조회할 수는 있습니다. 하지만 버그를 수정하려면 .py, .js 등 소스 파일 내용을 수정해야 하며, 새로운 테스트 파일을 생성할 때도 기존 디렉터리에 파일을 작성해야 하므로 파일 쓰기 권한이 추가로 필요합니다.

07-2 AI 에이전트로 소프트웨어 개발 자동화하기

1. **답** ① backend는 API 연동만, frontend는 전체 UI, QA는 모든 시나리오 테스트
2. **답** ④ PRD는 .claude/agents/ 폴더에 자동으로 복사되어 공유된다.

08-1 MCP 이해하고 클로드 코드와 연결하기

1. **답** ② QA 에이전트는 이전 에이전트들의 작업 결과를 모두 검토해야 하기 때문이다.
2. **답** ② 클로드 코드 옵션과 MCP 서버 실행 명령을 구분
3. **답** Sequential Thinking MCP는 문제를 순차적으로 분해하여 체계적으로 분석하며, 목표, 실행 방법, 로드맵, 측정 지표, 리스크 대응 방법 등을 상세히 제공하여 일반 추론보다 더 구조화되고 포괄적인 결과를 생성합니다.

08-2 MCP로 구현하는 완전 자동화 개발 환경

1. **답** ② .playwright-mcp
2. **답** ③ add → commit → push

- 3. 답** ① 구글에 MCP의 이름을 검색하고 깃허브 저장소에 접속해 MCP 서버 주소 찾기
 ② MCP 서버 목록이 정리된 깃허브 저장소나 웹사이트를 활용하여 카테고리별로 탐색하는 방법

08-3 데이터베이스 연결해 진짜 서비스 만들기

- 1. 답** ② 데이터베이스는 여러 기기에서 데이터를 공유할 수 있다.
- 2. 답** ② index.html 파일 부재 - 파일명 변경 필요
- 3. 답** • 유사점: 데이터베이스의 테이블은 엑셀 시트와 유사하게 행과 열로 구성된 표 형태로 데이터를 저장합니다.
 • 차이점: 데이터베이스의 테이블은 여러 사용자가 동시에 접근 가능하고, 실시간으로 동기화되며, 프로그램과 연동하여 자동으로 데이터를 처리할 수 있습니다.

ㄱ

거대 언어 모델 027

게시 040

계층 구조 119

그록 032

깃 086

깃 설치 088

깃허브 335

ㄴ

노드JS 086

노드JS 설치 086

노션 306

뉴럴 네트워크 031

ㄷ

단계별 프롬프트 전략 129

데이터베이스 346, 356

ㄹ

로컬 MCP 315

로컬 스토리지 356

로컬호스트 122

로컬 환경 254

ㄴ

멀티모달 AI 158

명령어 체이닝 210

명령어 폴더 195

명령 프롬프트 089

모델 선택 264

모델 선택형 029

ㄴ

바이브 코딩 027

배치 파일 107

백업 280

백엔드 개발자 284

버전 관리 323

분당 요청 횟수 233

ㄴ

생성적 사전 학습 트랜스포머 031

생성적 적대 신경망 031

서브에이전트 259, 285

세션 147

손글씨 인식 100

순차적 추론 MCP 318

순환 신경망 031

ㅇ

아티팩트 036

앤티로픽 031

어텐션 메커니즘 031

에이전트 258

오토인코더 031

오픈라우터 226

원격 MCP 315
 웹 프레임워크 247
 유료 플랜 053
 이중 하이픈 317
 인수 198
 인증 316

ㄷ

자동 승인 모드 150
 전용 모델형 029
 제미나이 032
 제품 기획 관리자 284
 종료 097

ㄹ

챗GPT 031
 최적화 267
 추가 342

ㄴ

커밋 342
 커스터마이징 044
 커스텀 명령어 194
 컨텍스트 153
 컨텍스트 관리 153
 클라우드 스토리지 356
 클라이언트-서버 모델 306
 클로드 030
 클로드 코드 030, 085

ㄷ

터미널 084
 테이블 362
 통합 커스텀 명령어 213
 트랜스포머 031

ㄹ

파이썬 104
 파일 권한 모드 109
 포트 122
 포트폴리오 068
 푸시 342
 품질 보증 엔지니어 284
 프론트엔드 개발자 284
 프롬프트 038, 058
 프롬프트 자료실 132
 플레이스홀더 198

ㅎ

할루시네이션 174
 합성곱 신경망 031
 환각 174

A

add 342
 AE 031
 agent 258
 AI assistant 031
 AI 어시스턴트 031

Anthropic 031
API 225
API key 227
API 키 227
API 토큰 337
Application Programming Interface 225
argument 198
artifacts 036
attention mechanism 031
authentication 316
Autoencoder 031

B

backend developer 284
backup 280
batch file 107

C

ChatGPT 031
Claude 030
Claude Code 030, 085
CLAUDE.md 114, 184
CLI 030
Client-Server Model 306
cloud storage 356
cmd 089
CNN 031
command chaining 210
Command Line Interface 030
command prompt 089

commit 342
context 153
Context7 MCP 326
Convolutional Neural Network 031
CS model 306
custom command 194
customize 044

D

database 346, 356
DB 346, 356
DeepSeek 235

F

Flask 247
frontend developer 284

G

GAN 031
Gemini 032
Gemma 234
Generative Adversarial Networks 031
Generative Pre-trained Transformer 031
GitHub 335
GPT 031
Gradio 247
Graphical User Interface 030
Grok 032
GUI 030

H

hallucination 174
hierarchical structure 119

L

Large Language Model 027
LLM 027
local environment 254
localhost 122
localStorage 356

M

MCP 305
MNIST 103
Model Context Protocol 305
Modified National Institute of Standards and
Technology 103
multimodal AI 158

N

neural network 031
Notion 306

O

OpenRouter 226
optimization 267

P

placeholder 198
Playwright MCP 327
PM 284
port 122
portfolio 068
PRD 062, 244
Product Manager 284
Product Requirements Document 062
prompt 038
push 342

Q

QA 284
Quality Assurance engineer 284

R

rate limit 233
Recurrent Neural Network 031
RNN 031

S

Sequential Thinking MCP 316
session 147
Streamlit 247
subagent 259
Supabase 356
Supabase MCP 358

T

table 362
terminal 084
token 337
transformer 031

U

UI 개선 162
User Experience Design 271
UX 디자인 271

V

Vercel 347
version control 323
vibe coding 027

W

web framework 247

기호

@ 164
&& 211
117
-c 149
/clear 186
/compact 155
/config 167
/context 154

--continue 149

/doctor 168

.env 231

/export 168

.gitignore 239

/help 144

/init 115

.json 142

/memory 182

/model 143

/resume 149

/status 143

숫자

5W1H 063, 289